App.tsx
538 lines

tsx

index.html
50 lines

html

index.tsx
17 lines

tsx

Dobrze — teraz już mamy **pełny obraz**. Na podstawie dostarczonego raportu **DTR STUDIO ENGINE** oraz Twojego realnego use-case'u (ebook + HTML karty pracy + multimedia + brak ingerencji w treść) poniżej **dopracowuję aplikację**, ale **nie jako opis marketingowy**, tylko jako **konkretną ewolucję

pasted

dopracuj app do perfekcji kroyko i zwiezle

18 sty
DTR STUDIO ENGINE v2.0 - ASSEMBLY-FIRST EVOLUTION
KRYTYCZNE ZMIANY KONCEPCYJNE
1. DUAL-MODE ARCHITECTURE
typescript
```
enum WorkflowMode {
  ASSEMBLY = 'assembly',  // Nowy tryb główny
  STUDIO = 'studio'       // Obecny tryb layoutu
}
```
ASSEMBLY MODE (nowy priorytet):

Zero AI ingerencji w treść
Manifest-driven injection
Placeholders → Assets mapping
Validation-first approach
STUDIO MODE (obecny):

Visual DNA dla layoutu
Typography orchestration
Print-ready DTP
2. MANIFEST-DRIVEN SYSTEM
Dodaj nowy moduł services/manifestService.ts:

typescript
```
interface AssemblyManifest {
  version: string;
```

```
  sections: {
    id: string;
    placeholder: string;  // {{WORKCARD_DAY_01}}
    type: 'html' | 'audio' | 'image';
    source: string;
    metadata?: Record<string, any>;
  }[];
}
```

Workflow:
```
1. Upload manifest.json
2. Parse placeholders
3. Validate assets presence
4. Inject deterministically
5. Export atomic
3. NOWA STRUKTURA KROKÓW
typescript
enum Step {
  MODE_SELECT = 'mode-select',           // NOWE

  // ASSEMBLY PATH
  MANIFEST_UPLOAD = 'manifest-upload',   // NOWE
  ASSET_MAPPING = 'asset-mapping',       // NOWE
  ASSEMBLY_VALIDATE = 'assembly-validate', // NOWE

  // STUDIO PATH (obecne)
  MANUSCRIPT_INGEST = 'manuscript-ingest',
  DTP_ORCHESTRATION = 'dtp-orchestration',
  ASSET_CONFIGURATION = 'asset-configuration',
  PREFLIGHT_PROOFING = 'preflight-proofing'
}
4. INLINE HTML INJECTION (killer feature)
Zamiast iframe → native injection:

typescript
interface WorkcardAsset extends ChapterAsset {
  type: 'workcard-html';
  inlineHTML: string;
  scopedCSS: string;
  placeholder: string; // {{WORKCARD_01}}
}
Rendering:

tsx
<section
  data-workcard-id={asset.id}
  dangerouslySetInnerHTML={{ __html: asset.inlineHTML }}
  style={{ isolation: 'isolate' }} // CSS sandboxing
/>
5. AI OGRANICZONY DO VALIDATION
Nowy plik services/assemblyValidator.ts:
```

```typescript
async function validateAssembly(manifest, assets) {
  return await callAI({
    prompt: `
      STRICT TASK: Validate assembly completeness.

      Manifest placeholders: ${JSON.stringify(manifest)}
      Available assets: ${JSON.stringify(assets)}

      Return JSON:
      {
        "missing": ["{{PLACEHOLDER}}", ...],
        "orphaned": ["asset_without_placeholder", ...],
        "status": "COMPLETE" | "INCOMPLETE"
      }

      DO NOT generate content.
      DO NOT suggest layouts.
      ONLY validate mapping.
    `
  });
}
```

Tokenów używane: ~90% mniej niż teraz.

6. SPLIT VIEW UI (Assembly Mode)

```tsx
<div className="grid grid-cols-2 gap-8">
  {/* LEFT: Structure Tree */}
  <div className="space-y-4">
    {manifest.sections.map(s => (
      <div className={placeholderMapped(s) ? 'bg-green-500/10' : 'bg-red-
500/10'}>
        <code>{s.placeholder}</code>
        {s.source && <CheckCircle2 />}
      </div>
    ))}
  </div>

  {/* RIGHT: Live Preview */}
  <iframe src={assemblyPreview} />
</div>
```

7. NOWY FLOW STARTOWY

```tsx
// MODE_SELECT screen
<div className="grid grid-cols-2 gap-20">
  <button onClick={() => setMode('assembly')}>
    <Command size={80} />
    <h2>ASSEMBLY MODE</h2>
    <p>Manifest-driven ebook assembly</p>
    <span>Perfect for: workbooks, courses, therapy</span>
  </button>

  <button onClick={() => setMode('studio')}>
    <Palette size={80} />
```

```
    <h2>STUDIO MODE</h2>
    <p>AI-powered layout orchestration</p>
    <span>Perfect for: novels, portfolios, magazines</span>
  </button>
</div>
```

8. MANIFEST.JSON SCHEMA

```
json
{
  "$schema": "https://dtr.studio/manifest.schema.json",
  "version": "2.0",
  "metadata": {
    "title": "30-Day Therapy Workbook",
    "author": "Dr. Example"
  },
  "sections": [
    {
      "id": "day-01",
      "chapter": 1,
      "placeholders": {
        "{{WORKCARD_01}}": "workcards/day01.html",
        "{{AUDIO_01}}": "audio/meditation01.mp3",
        "{{IMAGE_01}}": "images/nature01.jpg"
      }
    }
  ],
  "assets": {
    "css": "styles/workbook.css",
    "fonts": ["fonts/Inter.woff2"]
  }
}
```

9. VALIDATION DASHBOARD (nowy UI)

```
tsx
<div className="bg-slate-900 p-12 rounded-3xl">
  <h3>Assembly Validation Report</h3>

  <div className="space-y-4">
    <div className="flex justify-between">
      <span>Placeholders detected</span>
      <span className="font-black">{validation.total}</span>
    </div>
    <div className="flex justify-between">
      <span>Assets mapped</span>
      <span className="text-green-500">{validation.mapped}</span>
    </div>
    <div className="flex justify-between">
      <span>Missing assets</span>
      <span className="text-red-500">{validation.missing.length}</span>
    </div>
  </div>

  {validation.status === 'COMPLETE' && (
    <button className="w-full bg-green-500">
      Generate Final Ebook
    </button>
```

```
  )}
</div>
```

10. EXPORT ENGINE (deterministyczny)

```typescript
async function exportEbook(mode: 'html' | 'epub' | 'pdf') {
  const assembled = await assembleFromManifest(manifest, assets);

  switch(mode) {
    case 'html':
      return bundleHTML(assembled); // Single HTML + embedded assets
    case 'epub':
      return bundleEPUB(assembled); // EPUB3 spec
    case 'pdf':
      return workerPDF(assembled);  // Off-thread rendering
  }
}
```

IMPLEMENTACJA - PRIORYTET

Dodaj MODE_SELECT screen (1h)
Stwórz manifestService.ts (2h)
Zaimplementuj placeholder parser (3h)
Dodaj validation dashboard (2h)
Testuj na realnym workbook (4h)
Total MVP: ~12h pracy.

USP (NOWY)

"From Manuscript to Interactive Book - Without Touching Your Words"

Dlaczego to wygrywa:

Terapeuci: "Nie zmienisz mojej metodyki"
Edukatorzy: "Potrzebuję stabilności, nie kreatywności AI"
Autorzy: "Mój tekst jest świętością"

MONETYZACJA - NOWY TIER

ASSEMBLY PRO ($29/mo):

Unlimited manifest assemblies
Zero AI layout overhead
Batch processing API
White-label export
Target: Terapeuci, szkoły, wydawcy workbooków.

Gotowe do implementacji. Kod czy architektura najpierw?

architektura

18 sty

DTR STUDIO ENGINE v2.0 - ARCHITEKTURA TECHNICZNA

1. STRUKTURA KATALOGÓW (FINALNA)

```
dtr-studio-engine/
├── src/
│   ├── modes/                          # NOWE - Separation of Concerns
│   │   ├── assembly/
│   │   │   ├── AssemblyWorkflow.tsx
│   │   │   ├── ManifestUpload.tsx
```

```
│   │   │   ├── AssetMapper.tsx
│   │   │   ├── ValidationDashboard.tsx
│   │   │   └── AssemblyPreview.tsx
│   │   │
│   │   └── studio/
│   │       ├── StudioWorkflow.tsx
│   │       ├── ManuscriptIngest.tsx
│   │       ├── AssetConfiguration.tsx
│   │       └── PreflightProofing.tsx
│   │
│   ├── services/
│   │   ├── assembly/                   # NOWE - Assembly Engine
│   │   │   ├── manifestService.ts      # Parse & validate manifest
│   │   │   ├── placeholderParser.ts    # Detect {{PLACEHOLDERS}}
│   │   │   ├── assetInjector.ts        # Inject HTML/media
│   │   │   ├── assemblyValidator.ts    # AI-powered validation only
│   │   │   └── exportEngine.ts         # HTML/EPUB/PDF export
│   │   │
│   │   ├── studio/                     # Obecne serwisy
│   │   │   ├── geminiService.ts
│   │   │   ├── metadataService.ts
│   │   │   └── qrService.ts
│   │   │
│   │   └── shared/
│   │       ├── storageService.ts       # Unified storage
│   │       └── workerPool.ts           # Off-thread processing
│   │
│   ├── types/
│   │   ├── assembly.types.ts           # NOWE
│   │   ├── studio.types.ts             # Refactor z types.ts
│   │   └── shared.types.ts
│   │
│   ├── workers/                        # NOWE - Off-main-thread
│   │   ├── pdf.worker.ts
│   │   ├── epub.worker.ts
│   │   └── validation.worker.ts
│   │
│   ├── components/
│   │   ├── shared/
│   │   │   ├── ModeSelector.tsx        # NOWE - Entry point
│   │   │   ├── StepIndicator.tsx
│   │   │   └── Sidebar.tsx
│   │   │
│   │   └── assembly/
│   │       ├── PlaceholderTree.tsx     # NOWE
│   │       ├── AssetMappingPanel.tsx   # NOWE
│   │       └── ValidationReport.tsx    # NOWE
│   │
│   ├── schemas/
│   │   └── manifest.schema.json        # NOWE - JSON Schema validation
│   │
│   └── App.tsx                         # Refactored - mode router
│
├── public/
```

```
│   ├── examples/                       # NOWE - Example manifests
│   │   ├── therapy-workbook.json
│   │   ├── course-ebook.json
│   │   └── interactive-novel.json
│   │
│   └── templates/                      # Manifest templates
│       └── manifest.template.json
│
└── docs/
    ├── ASSEMBLY_GUIDE.md               # NOWE
    ├── MANIFEST_SPEC.md                # NOWE
    └── API.md
```

2. TYPE SYSTEM (assembly.types.ts)

typescript

```typescript
// ============ MANIFEST TYPES ============

export interface AssemblyManifest {
  $schema: string;
  version: '2.0';
  metadata: ManifestMetadata;
  sections: ManifestSection[];
  assets: GlobalAssets;
  config?: AssemblyConfig;
}

export interface ManifestMetadata {
  title: string;
  author: string;
  isbn?: string;
  language: string;
  genre?: string;
  description?: string;
}

export interface ManifestSection {
  id: string;
  chapter: number;
  title: string;
  placeholders: Record<string, string>; // {{KEY}}: "path/to/asset"
  metadata?: {
    wordCount?: number;
    duration?: number; // for audio
    interactivity?: 'static' | 'interactive';
  };
}

export interface GlobalAssets {
  css?: string[];
  fonts?: string[];
  scripts?: string[];
}

export interface AssemblyConfig {
  strict: boolean;           // Fail on missing assets?
```

```
  inlineCSS: boolean;        // Inline or external CSS?
  sandboxHTML: boolean;      // Isolate workcard styles?
  optimizeImages: boolean;   // Compress on export?
}

// ============ PROCESSING TYPES ============

export interface PlaceholderMapping {
  placeholder: string;       // {{WORKCARD_01}}
  type: AssetType;
  sourcePath: string;
  resolved: boolean;
  asset?: ProcessedAsset;
}

export interface ProcessedAsset {
  id: string;
  originalPath: string;
  processedURL: string;      // Blob URL or base64
  type: AssetType;
  size: number;
  checksum: string;          // For integrity validation
}

export interface ValidationResult {
  status: 'COMPLETE' | 'INCOMPLETE' | 'ERROR';
  total: number;
  mapped: number;
  missing: PlaceholderMapping[];
  orphaned: string[];        // Assets without placeholders
  errors: ValidationError[];
}

export interface ValidationError {
  type: 'MISSING_ASSET' | 'INVALID_PATH' | 'UNSUPPORTED_TYPE';
  placeholder: string;
  message: string;
  severity: 'ERROR' | 'WARNING';
}

// ============ EXPORT TYPES ============

export interface ExportConfig {
  format: 'html' | 'epub' | 'pdf';
  quality: 'draft' | 'print' | 'digital';
  includeMetadata: boolean;
  compression: boolean;
}

export interface ExportResult {
  success: boolean;
  format: string;
  size: number;
  downloadURL: string;
```

```
  metadata: {
    pages?: number;
    assets: number;
    generatedAt: string;
  };
}
```

3. MANIFEST SERVICE (manifestService.ts)

typescript

```
import Ajv from 'ajv';
import manifestSchema from '../schemas/manifest.schema.json';

const ajv = new Ajv();
const validateSchema = ajv.compile(manifestSchema);

export class ManifestService {

  // Parse i walidacja manifestu
  static async parseManifest(file: File): Promise<AssemblyManifest> {
    const text = await file.text();
    const data = JSON.parse(text);

    if (!validateSchema(data)) {
      throw new Error(`Invalid manifest:
${JSON.stringify(validateSchema.errors)}`);
    }

    return data as AssemblyManifest;
  }

  // Ekstrakcja wszystkich placeholderów
  static extractPlaceholders(manifest: AssemblyManifest):
PlaceholderMapping[] {
    const mappings: PlaceholderMapping[] = [];

    manifest.sections.forEach(section => {
      Object.entries(section.placeholders).forEach(([key, path]) => {
        mappings.push({
          placeholder: key,
          type: this.detectAssetType(path),
          sourcePath: path,
          resolved: false
        });
      });
    });

    return mappings;
  }

  // Detekcja typu assetu z rozszerzenia
  private static detectAssetType(path: string): AssetType {
    const ext = path.split('.').pop()?.toLowerCase();

    if (['html', 'htm'].includes(ext!)) return 'workcard-html';
    if (['mp3', 'wav', 'ogg'].includes(ext!)) return 'audio';
```

```
    if (['jpg', 'jpeg', 'png', 'webp'].includes(ext!)) return 'raster';
    if (['svg'].includes(ext!)) return 'vector';

    return 'source';
  }

  // Walidacja struktury (bez AI)
  static validateStructure(manifest: AssemblyManifest): ValidationError[]
{
    const errors: ValidationError[] = [];

    // Check sections order
    const chapters = manifest.sections.map(s => s.chapter);
    if (chapters.some((ch, i) => i > 0 && ch <= chapters[i-1])) {
      errors.push({
        type: 'INVALID_PATH',
        placeholder: 'sections',
        message: 'Chapters must be in ascending order',
        severity: 'ERROR'
      });
    }

    // Check duplicate placeholders
    const allPlaceholders = manifest.sections.flatMap(s =>
      Object.keys(s.placeholders)
    );
    const duplicates = allPlaceholders.filter((p, i) =>
      allPlaceholders.indexOf(p) !== i
    );

    if (duplicates.length > 0) {
      errors.push({
        type: 'INVALID_PATH',
        placeholder: duplicates[0],
        message: `Duplicate placeholder: ${duplicates[0]}`,
        severity: 'ERROR'
      });
    }

    return errors;
  }
}
```

4. PLACEHOLDER PARSER (placeholderParser.ts)

typescript

```
export class PlaceholderParser {

  // Wykryj wszystkie {{PLACEHOLDERS}} w tekście
  static detectInText(text: string): string[] {
    const regex = /\{\{([A-Z0-9_]+)\}\}/g;
    const matches = [...text.matchAll(regex)];
    return [...new Set(matches.map(m => `{{${m[1]}}}`))];
  }

  // Zamień placeholdery na rzeczywisty content
```

```
  static async injectContent(
    text: string,
    mappings: Map<string, ProcessedAsset>
  ): Promise<string> {
    let result = text;

    for (const [placeholder, asset] of mappings) {
      const replacement = await this.generateReplacement(asset);
      result = result.replaceAll(placeholder, replacement);
    }

    return result;
  }

  // Generuj HTML replacement w zależności od typu
  private static async generateReplacement(asset: ProcessedAsset):
Promise<string> {
    switch (asset.type) {
      case 'workcard-html':
        return `
          <section
            data-workcard-id="${asset.id}"
            data-checksum="${asset.checksum}"
            class="workcard-inject"
            style="isolation: isolate;"
          >
            ${await fetch(asset.processedURL).then(r => r.text())}
          </section>
        `;

      case 'audio':
        return `
          <audio
            controls
            data-asset-id="${asset.id}"
            class="audio-inject"
          >
            <source src="${asset.processedURL}" type="audio/mpeg">
          </audio>
        `;

      case 'raster':
        return `
          <figure data-asset-id="${asset.id}" class="image-inject">
            <img
              src="${asset.processedURL}"
              alt="Asset ${asset.id}"
              loading="lazy"
            />
          </figure>
        `;

      default:
        return `<!-- UNSUPPORTED ASSET TYPE: ${asset.type} -->`;
```

```
    }
  }
}
```

5. ASSEMBLY VALIDATOR (assemblyValidator.ts)

typescript

```
export class AssemblyValidator {

  // AI-powered semantic validation (TYLKO validation, zero content
generation)
  static async validateWithAI(
    manifest: AssemblyManifest,
    mappings: PlaceholderMapping[]
  ): Promise<ValidationResult> {

    const prompt = `
STRICT VALIDATION TASK - DO NOT GENERATE CONTENT

Manifest sections: ${manifest.sections.length}
Expected placeholders: ${mappings.length}

Placeholders status:
${mappings.map(m => `${m.placeholder}: ${m.resolved ? 'RESOLVED' :
'MISSING'}`).join('\n')}

YOUR TASK:
1. Identify missing assets
2. Detect orphaned files
3. Check naming consistency
4. Report errors

RETURN ONLY JSON:
{
  "status": "COMPLETE" | "INCOMPLETE",
  "missing": ["{{PLACEHOLDER}}", ...],
  "orphaned": ["filename.ext", ...],
  "warnings": ["message", ...]
}

DO NOT:
- Generate placeholder content
- Suggest layouts
- Modify manifest
- Create assets
`;

    const response = await fetch('https://api.anthropic.com/v1/messages',
{
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        model: 'claude-sonnet-4-20250514',
        max_tokens: 1000,
        messages: [{ role: 'user', content: prompt }]
      })
```

```
    });

    const data = await response.json();
    const result = JSON.parse(
      data.content[0].text.replace(/```json|```/g, '').trim()
    );

    return {
      status: result.status,
      total: mappings.length,
      mapped: mappings.filter(m => m.resolved).length,
      missing: mappings.filter(m => !m.resolved),
      orphaned: result.orphaned || [],
      errors: result.warnings?.map((w: string) => ({
        type: 'MISSING_ASSET',
        placeholder: '',
        message: w,
        severity: 'WARNING'
      })) || []
    };
  }

  // Lokalna walidacja (bez AI)
  static validateLocal(mappings: PlaceholderMapping[]): ValidationResult
{
    return {
      status: mappings.every(m => m.resolved) ? 'COMPLETE' :
'INCOMPLETE',
      total: mappings.length,
      mapped: mappings.filter(m => m.resolved).length,
      missing: mappings.filter(m => !m.resolved),
      orphaned: [],
      errors: []
    };
  }
}
```

6. EXPORT ENGINE (exportEngine.ts)

typescript

```
export class ExportEngine {

  // Export do HTML (single file)
  static async exportHTML(
    manifest: AssemblyManifest,
    assembledContent: string,
    assets: ProcessedAsset[]
  ): Promise<ExportResult> {

    const html = `
<!DOCTYPE html>
<html lang="${manifest.metadata.language}">
<head>
  <meta charset="UTF-8">
  <meta name="author" content="${manifest.metadata.author}">
  <title>${manifest.metadata.title}</title>
```

```
  <style>${await this.bundleCSS(manifest.assets.css || [])}</style>
</head>
<body>
  <main class="ebook-container">
    ${assembledContent}
  </main>
  <script>${await this.bundleScripts(manifest.assets.scripts ||
[])}</script>
</body>
</html>
    `;

    const blob = new Blob([html], { type: 'text/html' });
    const url = URL.createObjectURL(blob);

    return {
      success: true,
      format: 'html',
      size: blob.size,
      downloadURL: url,
      metadata: {
        assets: assets.length,
        generatedAt: new Date().toISOString()
      }
    };
  }

  // Export do EPUB (via worker)
  static async exportEPUB(
    manifest: AssemblyManifest,
    assembledContent: string,
    assets: ProcessedAsset[]
  ): Promise<ExportResult> {

    const worker = new Worker(new URL('../workers/epub.worker.ts',
import.meta.url));

    return new Promise((resolve, reject) => {
      worker.postMessage({
        manifest,
        content: assembledContent,
        assets
      });

      worker.onmessage = (e) => {
        if (e.data.success) {
          resolve(e.data);
        } else {
          reject(e.data.error);
        }
        worker.terminate();
      };
    });
  }
```

```
  // Export do PDF (via worker - heavy processing)
  static async exportPDF(
    manifest: AssemblyManifest,
    assembledContent: string,
    config: ExportConfig
  ): Promise<ExportResult> {

    const worker = new Worker(new URL('../workers/pdf.worker.ts',
import.meta.url));

    return new Promise((resolve, reject) => {
      worker.postMessage({
        manifest,
        content: assembledContent,
        config
      });

      worker.onmessage = (e) => {
        if (e.data.success) {
          resolve(e.data);
        } else {
          reject(e.data.error);
        }
        worker.terminate();
      };
    });
  }

  private static async bundleCSS(cssPaths: string[]): Promise<string> {
    const styles = await Promise.all(
      cssPaths.map(path => fetch(path).then(r => r.text()))
    );
    return styles.join('\n\n');
  }

  private static async bundleScripts(scriptPaths: string[]):
Promise<string> {
    const scripts = await Promise.all(
      scriptPaths.map(path => fetch(path).then(r => r.text()))
    );
    return scripts.join('\n\n');
  }
}
```

7. APP.TSX (refactored - mode router)

typescript

```
const App: React.FC = () => {
  const [mode, setMode] = useState<WorkflowMode | null>(null);

  // MODE SELECTION
  if (!mode) {
    return <ModeSelector onSelect={setMode} />;
  }
```

```
  // ASSEMBLY WORKFLOW
  if (mode === WorkflowMode.ASSEMBLY) {
    return <AssemblyWorkflow />;
  }

  // STUDIO WORKFLOW
  return <StudioWorkflow />;
};
```

---

8. DATA FLOW (ASSEMBLY MODE)

```
USER UPLOADS
  ↓
manifest.json + assets.zip
  ↓
ManifestService.parseManifest()
  ↓
PlaceholderParser.detectInText()
  ↓
User maps assets via UI
  ↓
AssemblyValidator.validateLocal()
  ↓
[OPTIONAL] AssemblyValidator.validateWithAI()
  ↓
PlaceholderParser.injectContent()
  ↓
ExportEngine.export{HTML|EPUB|PDF}()
  ↓
DOWNLOAD FINAL EBOOK
Kluczowe: AI wywoływane tylko raz, na końcu, do walidacji.
```

9. WORKER ARCHITECTURE (pdf.worker.ts example)

typescript

```
// pdf.worker.ts
import * as pdfjsLib from 'pdfjs-dist';

self.onmessage = async (e) => {
  const { manifest, content, config } = e.data;

  try {
    // Heavy PDF processing off main thread
    const pdfDoc = await generatePDF(content, config);
    const blob = await pdfDoc.output('blob');
    const url = URL.createObjectURL(blob);

    self.postMessage({
      success: true,
      format: 'pdf',
      size: blob.size,
      downloadURL: url,
```

```
      metadata: {
        pages: pdfDoc.internal.pages.length,
        generatedAt: new Date().toISOString()
      }
    });
  } catch (error) {
    self.postMessage({
      success: false,
      error: error.message
    });
  }
};
```

10. MANIFEST SCHEMA (manifest.schema.json)

```
json
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "https://dtr.studio/manifest.schema.json",
  "title": "DTR Assembly Manifest",
  "type": "object",
  "required": ["version", "metadata", "sections"],
  "properties": {
    "version": {
      "type": "string",
      "enum": ["2.0"]
    },
    "metadata": {
      "type": "object",
      "required": ["title", "author", "language"],
      "properties": {
        "title": { "type": "string" },
        "author": { "type": "string" },
        "language": { "type": "string", "pattern": "^[a-z]{2}$" },
        "isbn": { "type": "string" }
      }
    },
    "sections": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["id", "chapter", "title", "placeholders"],
        "properties": {
          "id": { "type": "string" },
          "chapter": { "type": "integer", "minimum": 1 },
          "title": { "type": "string" },
          "placeholders": {
            "type": "object",
            "patternProperties": {
              "^\\{\\{[A-Z0-9_]+\\}\\}$": { "type": "string" }
            }
          }
        }
      }
    }
  }
```

```
}
```

11. STORAGE STRATEGY

Local Development

typescript

```
// In-memory podczas edycji
const assemblyStore = new Map<string, ProcessedAsset>();

// Persist do localStorage tylko manifest
localStorage.setItem('dtr_manifest', JSON.stringify(manifest));
```

Production

typescript

```
// Claude Artifacts Storage API
await window.storage.set(
  `manifest:${projectId}`,
  JSON.stringify(manifest)
);

// Assets jako base64 chunks (max 5MB per key)
await window.storage.set(
  `asset:${assetId}:chunk0`,
  base64Chunk,
  false // personal data
);
```

12. PERFORMANCE TARGETS

Operacja	Target	Method
Parse manifest	<100ms	Sync JSON.parse + validation
Map 50 assets	<2s	Async parallel processing
Validate structure	<500ms	Local checks only
AI validation (optional)	<5s	Single API call
Inject placeholders	<3s	String replacement batch
Export HTML	<1s	Template + bundle
Export EPUB	<5s	Worker thread
Export PDF	<15s	Worker thread

13. ERROR HANDLING STRATEGY

typescript

```
class AssemblyError extends Error {
  constructor(
    public code: string,
    public severity: 'ERROR' | 'WARNING',
    message: string
  ) {
    super(message);
  }
}

// Usage
try {
  const manifest = await ManifestService.parseManifest(file);
} catch (e) {
  if (e instanceof AssemblyError) {
    if (e.severity === 'ERROR') {
      showErrorModal(e.message);
    } else {
      showWarningToast(e.message);
```

```
    }
  }
}
```

14. TESTING STRATEGY

typescript

```
// Unit tests
describe('ManifestService', () => {
  it('should parse valid manifest', () => {});
  it('should reject invalid schema', () => {});
  it('should extract placeholders', () => {});
});

// Integration tests
describe('Assembly Pipeline', () => {
  it('should complete full assembly workflow', () => {});
  it('should handle missing assets gracefully', () => {});
});

// E2E tests (Playwright)
test('User uploads manifest and exports HTML', async ({ page }) => {
  // ...
});
```

IMPLEMENTATION ROADMAP

Phase 1: Foundation (Week 1)
- Refactor folder structure
- Create type definitions
- Implement ManifestService
- Implement PlaceholderParser

Phase 2: Core Assembly (Week 2)
- Build AssetMapper UI
- Implement validation (local + AI)
- Create ValidationDashboard
- Implement HTML export

Phase 3: Advanced Export (Week 3)
- Create PDF worker
- Create EPUB worker
- Implement worker pool
- Add progress tracking

Phase 4: Polish (Week 4)
- Mode selector UI
- Example manifests
- Documentation
- Testing suite

Gotowe. Kod implementacji teraz czy dalsze dopracowanie?

da;szr dopracowanie

19 sty

DTR STUDIO ENGINE v2.0 - DALSZE DOPRACOWANIE ARCHITEKTURY

1. CRITICAL: SECURITY & SANDBOXING

Problem

Wstrzykiwanie user-generated HTML = XSS risk.

Rozwiązanie: Multi-layer Sandboxing

```typescript
// services/assembly/htmlSanitizer.ts

import DOMPurify from 'dompurify';

export class HTMLSanitizer {

  private static ALLOWED_TAGS = [
    'div', 'span', 'p', 'h1', 'h2', 'h3', 'h4', 'h5', 'h6',
    'ul', 'ol', 'li', 'table', 'tr', 'td', 'th',
    'input', 'textarea', 'label', 'button', 'select', 'option',
    'img', 'audio', 'video', 'canvas'
  ];

  private static ALLOWED_ATTRS = [
    'class', 'id', 'style', 'data-*',
    'type', 'value', 'placeholder', 'name',
    'src', 'alt', 'width', 'height'
  ];

  private static FORBIDDEN_PATTERNS = [
    /javascript:/gi,
    /on\w+=/gi,  // onclick, onerror, etc.
    /<script/gi,
    /<iframe/gi,
    /eval\(/gi
  ];

  // Sanitize HTML przed injection
  static sanitize(html: string, strict: boolean = true): string {

    // 1. Check for forbidden patterns
    for (const pattern of this.FORBIDDEN_PATTERNS) {
      if (pattern.test(html)) {
        throw new AssemblyError(
          'SECURITY_VIOLATION',
          'ERROR',
          `Forbidden pattern detected: ${pattern.source}`
        );
      }
    }

    // 2. DOMPurify clean
    const clean = DOMPurify.sanitize(html, {
      ALLOWED_TAGS: strict ? this.ALLOWED_TAGS : undefined,
      ALLOWED_ATTR: this.ALLOWED_ATTRS,
      ALLOW_DATA_ATTR: true,
      FORBID_TAGS: ['script', 'iframe', 'object', 'embed'],
      FORBID_ATTR: ['onerror', 'onload', 'onclick']
    });

    // 3. Wrap w sandboxed container
    return `
      <div
```

```
        class="workcard-sandbox"
        data-sanitized="true"
        style="
          all: initial;
          display: block;
          isolation: isolate;
          contain: layout style paint;
        "
      >
        ${clean}
      </div>
    `;
  }

  // CSS isolation via Shadow DOM (optional)
  static wrapInShadowDOM(html: string, css: string): string {
    return `
      <div class="shadow-host">
        <template shadowrootmode="open">
          <style>
            :host { all: initial; display: block; }
            ${css}
          </style>
          ${html}
        </template>
      </div>
    `;
  }
}
```

Implementacja w PlaceholderParser:

```
typescript
private static async generateReplacement(asset: ProcessedAsset):
Promise<string> {
  switch (asset.type) {
    case 'workcard-html':
      const rawHTML = await fetch(asset.processedURL).then(r =>
r.text());
      const sanitized = HTMLSanitizer.sanitize(rawHTML, true);

      return `
        <section
          data-workcard-id="${asset.id}"
          data-checksum="${asset.checksum}"
          class="workcard-inject"
        >
          ${sanitized}
        </section>
      `;
    // ...
  }
}
```

2. ASSET PREPROCESSING PIPELINE

Problem

User uploady w różnych formatach, rozmiarach, jakościach.

Rozwiązanie: Unified Asset Processor
typescript

```
// services/assembly/assetProcessor.ts

export class AssetProcessor {

  // Process uploaded file → ProcessedAsset
  static async process(
    file: File,
    type: AssetType,
    config: ProcessingConfig
  ): Promise<ProcessedAsset> {

    switch (type) {
      case 'workcard-html':
        return this.processHTML(file, config);
      case 'raster':
        return this.processImage(file, config);
      case 'audio':
        return this.processAudio(file, config);
      default:
        return this.processGeneric(file);
    }
  }

  // HTML processing
  private static async processHTML(
    file: File,
    config: ProcessingConfig
  ): Promise<ProcessedAsset> {

    const html = await file.text();

    // Extract inline CSS
    const cssMatch = html.match(/<style[^>]*>([\s\S]*?)<\/style>/gi);
    const inlineCSS = cssMatch ? cssMatch.join('\n') : '';

    // Remove <html>, <head>, <body> wrappers
    const bodyContent = html
      .replace(/<\/?html[^>]*>/gi, '')
      .replace(/<head[^>]*>[\s\S]*?<\/head>/gi, '')
      .replace(/<\/?body[^>]*>/gi, '');

    // Sanitize
    const sanitized = HTMLSanitizer.sanitize(bodyContent);

    // Generate blob
    const blob = new Blob([sanitized], { type: 'text/html' });
    const url = URL.createObjectURL(blob);

    return {
      id: this.generateAssetId(),
```

```
    originalPath: file.name,
    processedURL: url,
    type: 'workcard-html',
    size: blob.size,
    checksum: await this.generateChecksum(blob),
    metadata: {
      inlineCSS,
      sanitized: true
    }
  };
}

// Image optimization
private static async processImage(
  file: File,
  config: ProcessingConfig
): Promise<ProcessedAsset> {

  const img = await createImageBitmap(file);

  const canvas = new OffscreenCanvas(
    config.maxWidth || img.width,
    config.maxHeight || img.height
  );

  const ctx = canvas.getContext('2d')!;

  // Resize preserving aspect ratio
  const ratio = Math.min(
    canvas.width / img.width,
    canvas.height / img.height
  );

  const w = img.width * ratio;
  const h = img.height * ratio;

  ctx.drawImage(img, 0, 0, w, h);

  // Convert to blob
  const blob = await canvas.convertToBlob({
    type: config.format || 'image/webp',
    quality: config.quality || 0.85
  });

  const url = URL.createObjectURL(blob);

  return {
    id: this.generateAssetId(),
    originalPath: file.name,
    processedURL: url,
    type: 'raster',
    size: blob.size,
    checksum: await this.generateChecksum(blob),
    metadata: {
```

```
        originalSize: file.size,
        compressionRatio: ((1 - blob.size / file.size) * 100).toFixed(1)
+ '%',
        dimensions: { width: w, height: h }
      }
    };
  }

  // Audio normalization
  private static async processAudio(
    file: File,
    config: ProcessingConfig
  ): Promise<ProcessedAsset> {

    // Dla audio możemy tylko validate + convert format
    const audioContext = new AudioContext();
    const arrayBuffer = await file.arrayBuffer();
    const audioBuffer = await audioContext.decodeAudioData(arrayBuffer);

    // Optional: normalize volume, trim silence, etc.
    // For now just pass-through

    const url = URL.createObjectURL(file);

    return {
      id: this.generateAssetId(),
      originalPath: file.name,
      processedURL: url,
      type: 'audio',
      size: file.size,
      checksum: await this.generateChecksum(file),
      metadata: {
        duration: audioBuffer.duration,
        sampleRate: audioBuffer.sampleRate,
        channels: audioBuffer.numberOfChannels
      }
    };
  }

  // Checksum generation (integrity verification)
  private static async generateChecksum(blob: Blob): Promise<string> {
    const buffer = await blob.arrayBuffer();
    const hashBuffer = await crypto.subtle.digest('SHA-256', buffer);
    const hashArray = Array.from(new Uint8Array(hashBuffer));
    return hashArray.map(b => b.toString(16).padStart(2, '0')).join('');
  }

  private static generateAssetId(): string {
    return `asset_${Date.now()}_${Math.random().toString(36).substr(2,
9)}`;
  }
}

interface ProcessingConfig {
```

```
  maxWidth?: number;
  maxHeight?: number;
  quality?: number;
  format?: 'image/webp' | 'image/jpeg' | 'image/png';
}
```

3. INCREMENTAL VALIDATION (Real-time Feedback)

Problem

User mapuje 50+ assetów → czeka na końcową walidację → błąd → frustracja.

Rozwiązanie: Live Validation Stream

typescript

```
// services/assembly/incrementalValidator.ts

export class IncrementalValidator {

  private mappings = new Map<string, PlaceholderMapping>();
  private listeners: Array<(result: PartialValidationResult) => void> =
[];

  // Subscribe to validation updates
  onUpdate(callback: (result: PartialValidationResult) => void) {
    this.listeners.push(callback);
  }

  // Add/update single mapping
  async updateMapping(placeholder: string, asset: ProcessedAsset | null)
{

    const mapping = this.mappings.get(placeholder);
    if (!mapping) return;

    mapping.asset = asset;
    mapping.resolved = asset !== null;

    // Instant validation
    const validation = this.validateSingle(mapping);

    // Notify listeners
    this.notifyUpdate({
      placeholder,
      status: validation.status,
      errors: validation.errors,
      progress: this.calculateProgress()
    });
  }

  // Validate single mapping
  private validateSingle(mapping: PlaceholderMapping): ValidationResult {
    const errors: ValidationError[] = [];

    if (!mapping.resolved) {
      errors.push({
        type: 'MISSING_ASSET',
        placeholder: mapping.placeholder,
```

```
        message: `Asset not mapped for ${mapping.placeholder}`,
        severity: 'ERROR'
      });
    }

    if (mapping.asset) {
      // Type mismatch check
      const expectedType = this.inferExpectedType(mapping.placeholder);
      if (expectedType && mapping.asset.type !== expectedType) {
        errors.push({
          type: 'UNSUPPORTED_TYPE',
          placeholder: mapping.placeholder,
          message: `Expected ${expectedType}, got ${mapping.asset.type}`,
          severity: 'WARNING'
        });
      }

      // Size check
      if (mapping.asset.size > 10 * 1024 * 1024) { // 10MB
        errors.push({
          type: 'INVALID_PATH',
          placeholder: mapping.placeholder,
          message: `Asset size ${(mapping.asset.size / 1024 /
1024).toFixed(1)}MB exceeds 10MB limit`,
          severity: 'WARNING'
        });
      }
    }

    return {
      status: errors.some(e => e.severity === 'ERROR') ? 'INCOMPLETE' :
'COMPLETE',
      total: 1,
      mapped: mapping.resolved ? 1 : 0,
      missing: mapping.resolved ? [] : [mapping],
      orphaned: [],
      errors
    };
  }

  // Infer expected type from placeholder name
  private inferExpectedType(placeholder: string): AssetType | null {
    if (/WORKCARD|CARD|WORKSHEET/i.test(placeholder)) return 'workcard-
html';
    if (/AUDIO|SOUND|MUSIC/i.test(placeholder)) return 'audio';
    if (/IMAGE|IMG|PHOTO|PICTURE/i.test(placeholder)) return 'raster';
    return null;
  }

  // Calculate overall progress
  private calculateProgress(): number {
    const total = this.mappings.size;
    const resolved = Array.from(this.mappings.values()).filter(m =>
m.resolved).length;
```

```
    return total > 0 ? (resolved / total) * 100 : 0;
  }

  private notifyUpdate(result: PartialValidationResult) {
    this.listeners.forEach(cb => cb(result));
  }
}

interface PartialValidationResult {
  placeholder: string;
  status: 'COMPLETE' | 'INCOMPLETE';
  errors: ValidationError[];
  progress: number;
}
```

UI Integration:

typescript

```
// components/assembly/AssetMappingPanel.tsx

const validator = useRef(new IncrementalValidator());

useEffect(() => {
  validator.current.onUpdate((result) => {
    // Update UI real-time
    setValidationState(prev => ({
      ...prev,
      [result.placeholder]: result
    }));

    setProgress(result.progress);
  });
}, []);

const handleAssetDrop = async (placeholder: string, file: File) => {
  const processed = await AssetProcessor.process(file, detectType(file));
  await validator.current.updateMapping(placeholder, processed);
};
```

4. UNDO/REDO SYSTEM (Command Pattern)

Problem

User mapuje 50 assetów → pomyłka → musi zacząć od nowa.

Rozwiązanie: Command Stack

typescript

```
// services/assembly/commandStack.ts

interface Command {
  execute(): Promise<void>;
  undo(): Promise<void>;
  description: string;
}

export class CommandStack {

  private stack: Command[] = [];
```

```
  private pointer: number = -1;

  async execute(command: Command) {
    // Remove future commands if we're in the middle of history
    this.stack = this.stack.slice(0, this.pointer + 1);

    await command.execute();

    this.stack.push(command);
    this.pointer++;
  }

  async undo() {
    if (!this.canUndo()) return;

    await this.stack[this.pointer].undo();
    this.pointer--;
  }

  async redo() {
    if (!this.canRedo()) return;

    this.pointer++;
    await this.stack[this.pointer].execute();
  }

  canUndo(): boolean {
    return this.pointer >= 0;
  }

  canRedo(): boolean {
    return this.pointer < this.stack.length - 1;
  }

  getHistory(): string[] {
    return this.stack.map(c => c.description);
  }
}

// Example: MapAssetCommand
export class MapAssetCommand implements Command {

  constructor(
    private placeholder: string,
    private newAsset: ProcessedAsset | null,
    private oldAsset: ProcessedAsset | null,
    private mappings: Map<string, PlaceholderMapping>
  ) {}

  async execute() {
    const mapping = this.mappings.get(this.placeholder);
    if (mapping) {
      mapping.asset = this.newAsset;
      mapping.resolved = this.newAsset !== null;
```

```
    }
  }

  async undo() {
    const mapping = this.mappings.get(this.placeholder);
    if (mapping) {
      mapping.asset = this.oldAsset;
      mapping.resolved = this.oldAsset !== null;
    }
  }

  get description(): string {
    return `Map ${this.newAsset?.originalPath || 'null'} to
${this.placeholder}`;
  }
}
```

UI:

```
tsx
<div className="flex gap-2">
  <button
    onClick={() => commandStack.undo()}
    disabled={!commandStack.canUndo()}
    className="p-2 bg-slate-800 rounded-xl"
  >
    <Undo size={16} />
  </button>
  <button
    onClick={() => commandStack.redo()}
    disabled={!commandStack.canRedo()}
    className="p-2 bg-slate-800 rounded-xl"
  >
    <Redo size={16} />
  </button>

  {/* History dropdown */}
  <select className="text-xs bg-slate-800 rounded-xl px-3">
    {commandStack.getHistory().map((desc, i) => (
      <option key={i}>{desc}</option>
    ))}
  </select>
</div>
```

5. BATCH OPERATIONS (Mass Actions)

Problem

50 kart HTML → wszystkie w tym samym folderze → mapowanie 1-by-1 = nightmare.

Rozwiązanie: Smart Batch Mapper

```
typescript
// services/assembly/batchMapper.ts

export class BatchMapper {

  // Auto-map by naming convention
```

```
static autoMapByNaming(
  placeholders: PlaceholderMapping[],
  assets: File[]
): Map<string, File> {

  const mappings = new Map<string, File>();

  for (const placeholder of placeholders) {
    // Extract semantic parts
    // {{WORKCARD_DAY_01}} → "workcard", "day", "01"
    const parts = this.parsePlaceholder(placeholder.placeholder);

    // Find best matching file
    const match = assets.find(file => {
      const fileName = file.name.toLowerCase();
      return parts.every(part => fileName.includes(part));
    });

    if (match) {
      mappings.set(placeholder.placeholder, match);
    }
  }

  return mappings;
}

// Bulk upload folder
static async uploadFolder(
  folderInput: HTMLInputElement
): Promise<Map<string, File>> {

  const files = Array.from(folderInput.files || []);
  const organized = new Map<string, File>();

  files.forEach(file => {
    // Preserve folder structure
    const path = file.webkitRelativePath || file.name;
    organized.set(path, file);
  });

  return organized;
}

// Pattern-based mapping
static mapByPattern(
  placeholders: PlaceholderMapping[],
  assets: File[],
  pattern: string // e.g., "workcard_{day}.html"
): Map<string, File> {

  const mappings = new Map<string, File>();

  // Convert pattern to regex
  const regex = new RegExp(
```

```
      pattern.replace(/\{(\w+)\}/g, '(?<$1>\\w+)')
    );

    for (const asset of assets) {
      const match = asset.name.match(regex);
      if (!match || !match.groups) continue;

      // Find placeholder with matching groups
      const placeholder = placeholders.find(p => {
        const pGroups = this.parsePlaceholder(p.placeholder);
        return Object.values(match.groups!).every(v =>
          pGroups.includes(v.toLowerCase())
        );
      });

      if (placeholder) {
        mappings.set(placeholder.placeholder, asset);
      }
    }

    return mappings;
  }

  private static parsePlaceholder(placeholder: string): string[] {
    // {{WORKCARD_DAY_01}} → ["workcard", "day", "01"]
    return placeholder
      .replace(/[{}]/g, '')
      .toLowerCase()
      .split(/[_\s-]+/)
      .filter(Boolean);
  }
}
UI:

tsx
<div className="bg-slate-900 p-8 rounded-3xl">
  <h3 className="text-xs font-black uppercase mb-6">Batch Operations</h3>

  <div className="space-y-4">
    <button onClick={handleAutoMap} className="w-full py-4 bg-amber-500
rounded-xl">
      <Zap size={16} className="inline mr-2" />
      Auto-Map by Naming
    </button>

    <input
      type="file"
      webkitdirectory=""
      directory=""
      ref={folderInputRef}
      className="hidden"
      onChange={handleFolderUpload}
    />
```

```
    <button onClick={() => folderInputRef.current?.click()} className="w-
full py-4 bg-slate-800 rounded-xl">
      <FolderPlus size={16} className="inline mr-2" />
      Upload Entire Folder
    </button>

    <div>
      <label className="text-xs text-slate-500 block mb-2">Naming
Pattern</label>
      <input
        className="w-full bg-slate-800 rounded-xl px-4 py-3 text-sm"
        placeholder="workcard_{day}.html"
        value={pattern}
        onChange={e => setPattern(e.target.value)}
      />
      <button onClick={handlePatternMap} className="w-full mt-2 py-3 bg-
slate-800 rounded-xl text-xs">
        Apply Pattern
      </button>
    </div>
  </div>
</div>
```

6. EXPORT PRESETS (Template-based Export)

Problem

User eksportuje do różnych kanałów: Kindle, Apple Books, Web, PDF print.

Rozwiązanie: Export Profile System

typescript

```
// services/assembly/exportProfiles.ts

export interface ExportProfile {
  id: string;
  name: string;
  description: string;
  format: 'html' | 'epub' | 'pdf';
  config: {
    pageSize?: 'A4' | 'A5' | 'US-Letter' | 'Kindle' | 'iPad';
    margins?: { top: number; right: number; bottom: number; left: number
};
    fontSize?: number;
    lineHeight?: number;
    fontFamily?: string;
    colorMode?: 'color' | 'grayscale' | 'bw';
    compression?: 'none' | 'standard' | 'maximum';
    includeCSS?: boolean;
    includeJS?: boolean;
    embedFonts?: boolean;
    optimizeImages?: boolean;
    accessibility?: boolean;
  };
}

export const EXPORT_PROFILES: ExportProfile[] = [
  {
```

```
    id: 'kindle-standard',
    name: 'Kindle Standard',
    description: 'Optimized for Kindle devices and apps',
    format: 'epub',
    config: {
      pageSize: 'Kindle',
      fontSize: 16,
      lineHeight: 1.5,
      colorMode: 'grayscale',
      compression: 'maximum',
      embedFonts: false,
      optimizeImages: true,
      accessibility: true
    }
  },
  {
    id: 'apple-books',
    name: 'Apple Books',
    description: 'For iBooks/Apple Books store',
    format: 'epub',
    config: {
      pageSize: 'iPad',
      fontSize: 18,
      lineHeight: 1.6,
      colorMode: 'color',
      compression: 'standard',
      embedFonts: true,
      optimizeImages: true,
      accessibility: true
    }
  },
  {
    id: 'web-interactive',
    name: 'Web Interactive',
    description: 'Full HTML with JavaScript interactions',
    format: 'html',
    config: {
      includeCSS: true,
      includeJS: true,
      compression: 'none',
      optimizeImages: false,
      accessibility: true
    }
  },
  {
    id: 'print-ready-a4',
    name: 'Print Ready (A4)',
    description: 'High-quality PDF for professional printing',
    format: 'pdf',
    config: {
      pageSize: 'A4',
      margins: { top: 20, right: 15, bottom: 20, left: 15 },
      fontSize: 12,
      lineHeight: 1.4,
```

```
      colorMode: 'color',
      compression: 'none',
      embedFonts: true,
      optimizeImages: false
    }
  },
  {
    id: 'therapy-workbook',
    name: 'Therapy Workbook',
    description: 'Interactive PDF with form fields preserved',
    format: 'pdf',
    config: {
      pageSize: 'A4',
      margins: { top: 25, right: 20, bottom: 25, left: 20 },
      fontSize: 14,
      lineHeight: 1.6,
      colorMode: 'color',
      compression: 'standard',
      embedFonts: true,
      optimizeImages: true,
      accessibility: true
    }
  }
];
```

Enhanced Export Engine:

typescript

```
// exportEngine.ts (extended)

export class ExportEngine {

  static async exportWithProfile(
    manifest: AssemblyManifest,
    assembledContent: string,
    assets: ProcessedAsset[],
    profileId: string
  ): Promise<ExportResult> {

    const profile = EXPORT_PROFILES.find(p => p.id === profileId);
    if (!profile) throw new Error('Profile not found');

    // Apply profile transformations
    const transformed = await this.applyProfileTransforms(
      assembledContent,
      assets,
      profile
    );

    // Export based on format
    switch (profile.format) {
      case 'html':
        return this.exportHTML(manifest, transformed, assets);
      case 'epub':
```

```
        return this.exportEPUB(manifest, transformed, assets,
profile.config);
      case 'pdf':
        return this.exportPDF(manifest, transformed, profile.config);
    }
  }

  private static async applyProfileTransforms(
    content: string,
    assets: ProcessedAsset[],
    profile: ExportProfile
  ): Promise<string> {

    let transformed = content;

    // Font size adjustment
    if (profile.config.fontSize) {
      transformed = transformed.replace(
        /<body/,
        `<body style="font-size: ${profile.config.fontSize}px; line-
height: ${profile.config.lineHeight}"`
      );
    }

    // Color mode conversion
    if (profile.config.colorMode === 'grayscale') {
      transformed = transformed.replace(
        /<style>/,
        '<style>* { filter: grayscale(100%); }'
      );
    }

    // Image optimization
    if (profile.config.optimizeImages) {
      for (const asset of assets.filter(a => a.type === 'raster')) {
        const optimized = await this.optimizeImage(asset, profile);
        transformed = transformed.replaceAll(asset.processedURL,
optimized.processedURL);
      }
    }

    // Accessibility enhancements
    if (profile.config.accessibility) {
      transformed = this.addAccessibilityAttributes(transformed);
    }

    return transformed;
  }

  private static addAccessibilityAttributes(html: string): string {
    // Add ARIA labels, semantic HTML, etc.
    return html
      .replace(/<img/g, '<img role="img"')
      .replace(/<button/g, '<button role="button"')
```

```
      .replace(/<input/g, '<input aria-required="false"');
  }
}
UI:

tsx
<div className="grid grid-cols-2 gap-6">
  {EXPORT_PROFILES.map(profile => (
    <div
      key={profile.id}
      onClick={() => setSelectedProfile(profile)}
      className={`p-8 rounded-3xl border-2 cursor-pointer transition-all
${
        selectedProfile?.id === profile.id
          ? 'border-amber-500 bg-amber-500/10'
          : 'border-slate-700 hover:border-slate-600'
      }`}
    >
      <h4 className="font-black text-lg mb-2">{profile.name}</h4>
      <p className="text-xs text-slate-500 mb-
4">{profile.description}</p>
      <div className="flex gap-2 text-[10px]">
        <span className="px-3 py-1 bg-slate-800 rounded-
full">{profile.format.toUpperCase()}</span>
        <span className="px-3 py-1 bg-slate-800 rounded-
full">{profile.config.pageSize || 'Responsive'}</span>
      </div>
    </div>
  ))}
</div>

<button
  onClick={() => handleExport(selectedProfile!.id)}
  disabled={!selectedProfile}
  className="w-full mt-8 py-6 bg-amber-500 rounded-3xl font-black text-lg
uppercase"
>
  Export as {selectedProfile?.name}
</button>
7. VERSION CONTROL & SNAPSHOTS
Problem
User iteruje nad projektem → chce wrócić do wcześniejszej wersji.

Rozwiązanie: Project Snapshots
typescript
// services/assembly/snapshotManager.ts

interface ProjectSnapshot {
  id: string;
  timestamp: string;
  manifest: AssemblyManifest;
  mappings: Map<string, ProcessedAsset>;
  metadata: {
    description?: string;
```

```
    autoSave: boolean;
  };
}

export class SnapshotManager {

  private static STORAGE_PREFIX = 'dtr_snapshot:';

  // Create snapshot
  static async createSnapshot(
    manifest: AssemblyManifest,
    mappings: Map<string, ProcessedAsset>,
    description?: string
  ): Promise<string> {

    const snapshot: ProjectSnapshot = {
      id: `snap_${Date.now()}`,
      timestamp: new Date().toISOString(),
      manifest,
      mappings,
      metadata: {
        description,
        autoSave: !description // Auto-save if no description provided
      }
    };

    // Serialize mappings (convert to plain object)
    const serialized = {
      ...snapshot,
      mappings: Array.from(snapshot.mappings.entries())
    };

    // Save to storage
    await window.storage.set(
      this.STORAGE_PREFIX + snapshot.id,
      JSON.stringify(serialized),
      false // personal data
    );

    return snapshot.id;
  }

  // Load snapshot
  static async loadSnapshot(id: string): Promise<ProjectSnapshot | null>
{
    try {
      const result = await window.storage.get(this.STORAGE_PREFIX + id);
      if (!result) return null;

      const data = JSON.parse(result.value);

      // Reconstruct Map
      return {
        ...data,
```

```
        mappings: new Map(data.mappings)
      };
    } catch {
      return null;
    }
  }

  // List all snapshots for project
Continue

19 sty
static async listSnapshots(manifestTitle: string):
Promise<ProjectSnapshot[]> {
const keys = await window.storage.list(this.STORAGE_PREFIX);
if (!keys) return [];

const snapshots = await Promise.all(
  keys.keys.map(key => this.loadSnapshot(key.replace(this.STORAGE_PREFIX,
'')))
);

return snapshots
  .filter((s): s is ProjectSnapshot => s !== null)
  .filter(s => s.manifest.metadata.title === manifestTitle)
  .sort((a, b) => new Date(b.timestamp).getTime() - new
Date(a.timestamp).getTime());
}

// Auto-save (throttled) static async autoSave( manifest:
AssemblyManifest, mappings: Map<string, ProcessedAsset> ): Promise<void>
{

// Delete old auto-saves (keep last 5)
const existing = await this.listSnapshots(manifest.metadata.title);
const autoSaves = existing.filter(s => s.metadata.autoSave);

if (autoSaves.length >= 5) {
  await window.storage.delete(
    this.STORAGE_PREFIX + autoSaves[autoSaves.length - 1].id
  );
}

await this.createSnapshot(manifest, mappings);
}
}


**UI:**
```tsx
<div className="bg-slate-900 p-6 rounded-3xl">
  <div className="flex justify-between items-center mb-6">
    <h3 className="text-xs font-black uppercase">Version History</h3>
    <button onClick={handleCreateSnapshot} className="text-amber-500
text-xs">
```
```

```
      + New Snapshot
    </button>
  </div>

  <div className="space-y-3 max-h-80 overflow-y-auto custom-scrollbar">
    {snapshots.map(snap => (
      <div
        key={snap.id}
        onClick={() => handleLoadSnapshot(snap.id)}
        className="p-4 bg-slate-800/50 rounded-2xl cursor-pointer
hover:bg-slate-800 transition-all"
      >
        <div className="flex justify-between items-start mb-2">
          <span className="text-xs font-black text-white">
            {snap.metadata.description || 'Auto-save'}
          </span>
          {snap.metadata.autoSave && (
            <History size={12} className="text-slate-600" />
          )}
        </div>
        <span className="text-[10px] text-slate-500">
          {new Date(snap.timestamp).toLocaleString()}
        </span>
      </div>
    ))}
  </div>
</div>
```

---

8. COLLABORATION FEATURES (Optional, ale killer)

Problem
Terapeuci współpracują z grafikami/edytorami.

Rozwiązanie: Shared Projects
```typescript
// types/assembly.types.ts (extended)

export interface CollaborationSession {
  projectId: string;
  owner: string;
  collaborators: Array<{
    id: string;
    role: 'editor' | 'viewer' | 'reviewer';
    joinedAt: string;
  }>;
  sharedAt: string;
}

// services/assembly/collaborationService.ts

export class CollaborationService {
```

```
  // Share project
  static async shareProject(
    manifest: AssemblyManifest,
    mappings: Map<string, ProcessedAsset>
  ): Promise<string> {

    const projectId = `proj_${Date.now()}`;

    // Save to shared storage
    await window.storage.set(
      `project:${projectId}:manifest`,
      JSON.stringify(manifest),
      true // shared = true
    );

    await window.storage.set(
      `project:${projectId}:mappings`,
      JSON.stringify(Array.from(mappings.entries())),
      true
    );

    // Generate share URL
    const shareUrl = `${window.location.origin}?project=${projectId}`;

    return shareUrl;
  }

  // Load shared project
  static async loadSharedProject(projectId: string): Promise<{
    manifest: AssemblyManifest;
    mappings: Map<string, ProcessedAsset>;
  } | null> {

    try {
      const manifestResult = await
window.storage.get(`project:${projectId}:manifest`, true);
      const mappingsResult = await
window.storage.get(`project:${projectId}:mappings`, true);

      if (!manifestResult || !mappingsResult) return null;

      return {
        manifest: JSON.parse(manifestResult.value),
        mappings: new Map(JSON.parse(mappingsResult.value))
      };
    } catch {
      return null;
    }
  }

  // Add comment to specific section
  static async addComment(
    projectId: string,
```

```
    sectionId: string,
    comment: string,
    author: string
  ): Promise<void> {

    const key = `project:${projectId}:comments:${sectionId}`;
    const existing = await window.storage.get(key, true);

    const comments = existing ? JSON.parse(existing.value) : [];
    comments.push({
      id: `comment_${Date.now()}`,
      text: comment,
      author,
      timestamp: new Date().toISOString()
    });

    await window.storage.set(key, JSON.stringify(comments), true);
  }
}
```

UI:
```tsx
<div className="bg-slate-900 p-6 rounded-3xl">
  <h3 className="text-xs font-black uppercase mb-4">Share Project</h3>

  <button
    onClick={handleShare}
    className="w-full py-4 bg-amber-500 rounded-xl mb-4"
  >
    <ExternalLink size={16} className="inline mr-2" />
    Generate Share Link
  </button>

  {shareUrl && (
    <div className="p-4 bg-slate-800 rounded-xl">
      <input
        readOnly
        value={shareUrl}
        className="w-full bg-transparent text-xs text-amber-500 outline-
none"
        onClick={(e) => e.currentTarget.select()}
      />
      <button
        onClick={() => navigator.clipboard.writeText(shareUrl)}
        className="mt-2 text-[10px] text-slate-500 hover:text-white"
      >
        Copy to Clipboard
      </button>
    </div>
  )}

  <div className="mt-6 pt-6 border-t border-slate-800">
```

```
    <h4 className="text-[10px] font-black uppercase text-slate-500 mb-
3">Active Collaborators</h4>
    {/* List collaborators */}
  </div>
</div>
```

---

9. PERFORMANCE MONITORING

```typescript
// services/monitoring/performanceMonitor.ts

export class PerformanceMonitor {

  private static metrics: Map<string, number[]> = new Map();

  static measure<T>(operation: string, fn: () => T | Promise<T>): T |
Promise<T> {
    const start = performance.now();

    const result = fn();

    if (result instanceof Promise) {
      return result.finally(() => {
        this.recordMetric(operation, performance.now() - start);
      }) as Promise<T>;
    } else {
      this.recordMetric(operation, performance.now() - start);
      return result;
    }
  }

  private static recordMetric(operation: string, duration: number) {
    if (!this.metrics.has(operation)) {
      this.metrics.set(operation, []);
    }
    this.metrics.get(operation)!.push(duration);

    // Keep last 100 measurements
    if (this.metrics.get(operation)!.length > 100) {
      this.metrics.get(operation)!.shift();
    }
  }

  static getStats(operation: string) {
    const measurements = this.metrics.get(operation) || [];
    if (measurements.length === 0) return null;

    const sorted = [...measurements].sort((a, b) => a - b);
    return {
      min: sorted[0],
      max: sorted[sorted.length - 1],
      avg: measurements.reduce((a, b) => a + b) / measurements.length,
```

```
      p50: sorted[Math.floor(sorted.length * 0.5)],
      p95: sorted[Math.floor(sorted.length * 0.95)],
      count: measurements.length
    };
  }

  static getAllStats() {
    return Array.from(this.metrics.keys()).map(op => ({
      operation: op,
      ...this.getStats(op)
    }));
  }
}

// Usage
const manifest = await PerformanceMonitor.measure(
  'manifest_parse',
  () => ManifestService.parseManifest(file)
);
```

---

10. FINAL ARCHITECTURE DIAGRAM

```
┌──────────────────────────────────────────────────────────────────────┐
│                        DTR STUDIO ENGINE v2.0                        │
│                        ASSEMBLY-FIRST ARCHITECTURE                   │
└──────────────────────────────────────────────────────────────────────┘


┌──────────────┐
│ MODE SELECT  │
└──────┬───────┘
       │
       ├──────────────────────────────────────────┬──────────────────────┐
       │                                          │                      │
       v                                          v                      v
┌──────────────┐                          ┌──────────────┐        ┌──────────────┐
│  ASSEMBLY    │                          │   STUDIO     │        │ SHARED CORE  │
│   MODE       │                          │   MODE       │        │  SERVICES    │
└──────┬───────┘                          └──────┬───────┘        └──────┬───────┘
       │                                         │                       │
       ├──────┬──────┬──────┬──────┐             ├──────┬──────┬──────┬──────┐       ├──────┬──────┬──────┬──────┐
       │      │      │      │      │             │      │      │      │      │       │      │      │      │      │
┌──────┴──────┴──────┴──────┴──────┐      ┌──────┴──────┴──────┴──────┴──────┐   ┌──────┴──────┴──────┴──────┴──────┐
│ Manifest Service                  │      │ Manuscript Ingest                 │   │ Storage API                       │
├───────────────────────────────────┤      ├───────────────────────────────────┤   ├───────────────────────────────────┤
│ Asset Processor                   │      │ AI Layout Orchestr.               │   │ Worker Pool                       │
├───────────────────────────────────┤      ├───────────────────────────────────┤   ├───────────────────────────────────┤
│ Placeholder Parser                │      │ Visual DNA Matrix                 │   │ Command Stack                     │
├───────────────────────────────────┤      ├───────────────────────────────────┤   ├───────────────────────────────────┤
│ Incremental Validator             │      │ Typography Engine                 │   │ Snapshot Manager                  │
```

```
┌──────────────┐        ┌──────────────┐        ┌──────────────┐
│ Batch        │        │ Asset        │        │ Export       │
│ Mapper       │        │ Config       │        │ Profiles     │
├──────────────┤        ├──────────────┤        ├──────────────┤
│ HTML         │        │ Preflight    │        │ Security     │
│ Sanitizer    │        │ Proofing     │        │ Sandbox      │
└──────────────┘        └──────────────┘        └──────────────┘
│                       │                       │
└───────────────────────┴───────────────────────┘
│
v
┌──────────────┐
│ EXPORT ENGINE │
├──────────────┤
│ HTML Worker  │
│ EPUB Worker  │
│ PDF Worker   │
└──────────────┘
│
v
┌──────────────┐
│ FINAL OUTPUT │
│ • ebook.html │
│ • ebook.epub │
│ • ebook.pdf  │
└──────────────┘
```

NASTĘPNE KROKI

Teraz możemy:

1. **Implementować kod** → zaczynamy od którego modułu?
2. **Zrobić mockupy UI** → pokażę Ci wizualizację Assembly Mode
3. **Napisać dokumentację techniczną** → dla devów
4. **Przygotować example manifests** → workbook terapeutyczny, kurs online
5. **Zdefiniować API contracts** → jeśli planujesz backend

Co teraz?
3

19 sty
DTR STUDIO ENGINE v2.0 - DOKUMENTACJA TECHNICZNA
SPIS TREŚCI
Wprowadzenie
Architektura Systemu
Manifest Specification
Core Services API
Security & Sandboxing
Workflow Guides
Performance Guidelines

Testing Strategy
Deployment
Troubleshooting
1. WPROWADZENIE
1.1 Czym jest DTR Studio Engine?
DTR Studio Engine v2.0 to silnik składu i publikacji ebooków interaktywnych z architekturą Assembly-First. System umożliwia deterministyczne łączenie treści, kart pracy HTML, multimediów i zasobów graficznych w kompletne publikacje cyfrowe.

Kluczowe cechy:

Zero ingerencji AI w treść autorską
Manifest-driven assembly (pełna kontrola)
Sandboxed HTML injection (bezpieczeństwo)
Multi-format export (HTML/EPUB/PDF)
Real-time validation
Batch operations
1.2 Dla kogo?

Grupa użytkowników	Use case	Tryb
Terapeuci	30-dniowe workbooki z kartami ćwiczeń	Assembly
Edukatorzy	Kursy online z interaktywnymi lekcjami	Assembly
Autorzy	Ebooki z materiałami dodatkowymi	Assembly
Designerzy	Portfolia, magazyny, publikacje wizualne	Studio
Wydawcy	Print-ready książki z DTP	Studio

1.3 Terminologia

Termin	Definicja
Manifest	Plik JSON definiujący strukturę projektu i mapowania assetów
Placeholder	Znacznik {{NAZWA}} w tekście, który zostanie zastąpiony assetem
Asset	Zasób: obraz, HTML, audio, video
Assembly	Proces łączenia manifestu z assetami w finalną publikację
Workcard	Interaktywna karta HTML (formularz, quiz, ćwiczenie)
DNA Template	Preset wizualny (typografia, layout, kolory)
Sandbox	Izolowane środowisko wykonania user-generated HTML

2. ARCHITEKTURA SYSTEMU
2.1 Separacja Trybów (Dual-Mode Architecture)
typescript
```
enum WorkflowMode {
  ASSEMBLY = 'assembly',  // Manifest-driven, zero AI content
  STUDIO = 'studio'       // AI-powered layout orchestration
}
```

ASSEMBLY MODE:
- Input: `manifest.json` + assets
- Process: Deterministyczny mapping + validation
- AI: Tylko opcjonalna walidacja (nie modyfikuje content)
- Output: Atomic export (HTML/EPUB/PDF)

STUDIO MODE:
- Input: Manuscript (DOCX/MD/TXT)
- Process: AI analiza → layout generation → visual orchestration
- AI: Gemini dla strukturyzacji + typography

- Output: Print-ready DTP

2.2 Warstwy Systemu (Layers)

```
┌──────────────────────────────────────────┐
│          PRESENTATION LAYER              │  React UI, Components
├──────────────────────────────────────────┤
│          APPLICATION LAYER               │  Workflows, State Management
├──────────────────────────────────────────┤
│          BUSINESS LOGIC LAYER            │  Services, Processors,
Validators
├──────────────────────────────────────────┤
│          DATA ACCESS LAYER               │  Storage API, File I/O
├──────────────────────────────────────────┤
│          INFRASTRUCTURE LAYER            │  Workers, Security,
Monitoring
└──────────────────────────────────────────┘
```

2.3 Struktura Plików (File Tree)

```
src/
├── modes/                      # Workflow implementations
│   ├── assembly/
│   │   ├── AssemblyWorkflow.tsx
│   │   ├── ManifestUpload.tsx
│   │   ├── AssetMapper.tsx
│   │   ├── ValidationDashboard.tsx
│   │   └── AssemblyPreview.tsx
│   └── studio/
│       ├── StudioWorkflow.tsx
│       ├── ManuscriptIngest.tsx
│       └── PreflightProofing.tsx
│
├── services/
│   ├── assembly/               # Assembly-specific services
│   │   ├── manifestService.ts
│   │   ├── placeholderParser.ts
│   │   ├── assetProcessor.ts
│   │   ├── assetInjector.ts
│   │   ├── assemblyValidator.ts
│   │   ├── batchMapper.ts
│   │   ├── htmlSanitizer.ts
│   │   ├── exportEngine.ts
│   │   ├── exportProfiles.ts
│   │   ├── snapshotManager.ts
│   │   ├── collaborationService.ts
│   │   └── incrementalValidator.ts
│   │
│   ├── studio/                 # Studio-specific services
│   │   ├── geminiService.ts
│   │   ├── metadataService.ts
│   │   └── qrService.ts
│   │
```

```
│   ├── shared/                 # Cross-mode utilities
│   │   ├── storageService.ts
│   │   ├── workerPool.ts
│   │   └── commandStack.ts
│   │
│   └── monitoring/
│       └── performanceMonitor.ts
│
├── workers/                    # Web Workers (off-main-thread)
│   ├── pdf.worker.ts
│   ├── epub.worker.ts
│   └── validation.worker.ts
│
├── types/
│   ├── assembly.types.ts
│   ├── studio.types.ts
│   └── shared.types.ts
│
├── components/
│   ├── shared/
│   │   ├── ModeSelector.tsx
│   │   ├── StepIndicator.tsx
│   │   └── Sidebar.tsx
│   └── assembly/
│       ├── PlaceholderTree.tsx
│       ├── AssetMappingPanel.tsx
│       └── ValidationReport.tsx
│
├── schemas/
│   └── manifest.schema.json    # JSON Schema for validation
│
└── App.tsx                     # Mode router
```

3. MANIFEST SPECIFICATION

3.1 Schema Overview

Manifest to single source of truth dla Assembly Mode. Definiuje:

Metadane publikacji
Strukturę rozdziałów/sekcji
Mapowanie placeholderów → assetów
Globalne zasoby (CSS, fonty, JS)
Konfigurację assembly

3.2 Full Schema (manifest.schema.json)

```
json
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "$id": "https://dtr.studio/manifest.schema.json",
  "title": "DTR Assembly Manifest v2.0",
  "type": "object",
  "required": ["version", "metadata", "sections"],

  "properties": {

    "version": {
      "type": "string",
```

```
    "enum": ["2.0"],
    "description": "Manifest schema version"
  },

  "metadata": {
    "type": "object",
    "required": ["title", "author", "language"],
    "properties": {
      "title": {
        "type": "string",
        "minLength": 1,
        "maxLength": 200
      },
      "author": {
        "type": "string",
        "minLength": 1,
        "maxLength": 100
      },
      "language": {
        "type": "string",
        "pattern": "^[a-z]{2}(-[A-Z]{2})?$",
        "examples": ["en", "pl", "en-US"]
      },
      "isbn": {
        "type": "string",
        "pattern": "^(97(8|9))?\\d{9}(\\d|X)$"
      },
      "genre": { "type": "string" },
      "description": {
        "type": "string",
        "maxLength": 2000
      },
      "coverImage": { "type": "string" },
      "publicationDate": {
        "type": "string",
        "format": "date"
      }
    }
  },

  "sections": {
    "type": "array",
    "minItems": 1,
    "items": {
      "type": "object",
      "required": ["id", "chapter", "title", "placeholders"],
      "properties": {

        "id": {
          "type": "string",
          "pattern": "^[a-z0-9-_]+$",
          "description": "Unique section identifier"
        },
```

```
      "chapter": {
        "type": "integer",
        "minimum": 1,
        "description": "Chapter number (must be sequential)"
      },

      "title": {
        "type": "string",
        "minLength": 1,
        "maxLength": 200
      },

      "placeholders": {
        "type": "object",
        "patternProperties": {
          "^\\{\\{[A-Z0-9_]+\\}\\}$": {
            "type": "string",
            "description": "Path to asset file"
          }
        },
        "additionalProperties": false
      },

      "metadata": {
        "type": "object",
        "properties": {
          "wordCount": { "type": "integer" },
          "duration": { "type": "integer" },
          "interactivity": {
            "enum": ["static", "interactive"]
          },
          "tags": {
            "type": "array",
            "items": { "type": "string" }
          }
        }
      }
    }
  }
},

"assets": {
  "type": "object",
  "properties": {
    "css": {
      "type": "array",
      "items": { "type": "string" },
      "description": "Global CSS files"
    },
    "fonts": {
      "type": "array",
      "items": { "type": "string" },
      "description": "Font files (WOFF2 recommended)"
    },
```

```
        "scripts": {
          "type": "array",
          "items": { "type": "string" },
          "description": "Global JS files"
        }
      }
    },

    "config": {
      "type": "object",
      "properties": {
        "strict": {
          "type": "boolean",
          "default": true,
          "description": "Fail assembly on missing assets?"
        },
        "inlineCSS": {
          "type": "boolean",
          "default": true,
          "description": "Inline CSS vs external link"
        },
        "sandboxHTML": {
          "type": "boolean",
          "default": true,
          "description": "Isolate workcard styles?"
        },
        "optimizeImages": {
          "type": "boolean",
          "default": true,
          "description": "Compress images on export?"
        },
        "accessibility": {
          "type": "boolean",
          "default": true,
          "description": "Add ARIA attributes?"
        }
      }
    }
  }
}
```

3.3 Example: Therapy Workbook Manifest

```
json
{
  "$schema": "https://dtr.studio/manifest.schema.json",
  "version": "2.0",

  "metadata": {
    "title": "30-Day Mindfulness Therapy Workbook",
    "author": "Dr. Anna Kowalska",
    "language": "pl",
    "isbn": "9788312345678",
    "genre": "Self-help / Psychology",
    "description": "Kompleksowy program terapii mindfulness z
interaktywnymi ćwiczeniami dziennymi.",
```

```
    "coverImage": "assets/cover.jpg",
    "publicationDate": "2025-02-01"
  },

  "sections": [
    {
      "id": "day-01",
      "chapter": 1,
      "title": "Dzień 1: Świadomy Oddech",
      "placeholders": {
        "{{WORKCARD_01}}": "workcards/day01_breathing.html",
        "{{AUDIO_01}}": "audio/meditation_breathing.mp3",
        "{{IMAGE_01}}": "images/nature_calm.jpg"
      },
      "metadata": {
        "duration": 15,
        "interactivity": "interactive",
        "tags": ["oddech", "podstawy", "mindfulness"]
      }
    },
    {
      "id": "day-02",
      "chapter": 2,
      "title": "Dzień 2: Skanowanie Ciała",
      "placeholders": {
        "{{WORKCARD_02}}": "workcards/day02_bodyscan.html",
        "{{AUDIO_02}}": "audio/meditation_bodyscan.mp3",
        "{{IMAGE_02}}": "images/body_awareness.jpg"
      },
      "metadata": {
        "duration": 20,
        "interactivity": "interactive",
        "tags": ["ciało", "świadomość", "relaksacja"]
      }
    }
    // ... days 3-30
  ],

  "assets": {
    "css": ["styles/workbook.css", "styles/forms.css"],
    "fonts": ["fonts/Inter-Variable.woff2", "fonts/Crimson-Pro.woff2"],
    "scripts": ["scripts/form-validation.js"]
  },

  "config": {
    "strict": true,
    "inlineCSS": true,
    "sandboxHTML": true,
    "optimizeImages": true,
    "accessibility": true
  }
}
```

3.4 Placeholder Naming Conventions

Konwencja: {{TYPE_DESCRIPTOR_INDEX}}

```
Type  Examples   Use case
WORKCARD   {{WORKCARD_DAY_01}}    Interaktywne karty HTML
AUDIO {{AUDIO_MEDITATION_01}}      Pliki dźwiękowe
IMAGE {{IMAGE_NATURE_01}}    Obrazy, fotografie
VIDEO {{VIDEO_TUTORIAL_01}}  Multimedia wideo
CHART {{CHART_PROGRESS_01}}  Wykresy, diagramy
QUIZ  {{QUIZ_KNOWLEDGE_01}}  Quizy, testy
Best practices:

Używaj UPPERCASE
Separator: underscore _
Numeracja: zawsze 2 cyfry (01, 02, ..., 99)
Opisowe, ale zwięzłe
4. CORE SERVICES API
4.1 ManifestService
typescript
class ManifestService {

  /**
   * Parse i walidacja manifestu
   * @throws AssemblyError jeśli schema invalid
   */
  static async parseManifest(file: File): Promise<AssemblyManifest>

  /**
   * Ekstrakcja wszystkich placeholderów
   * @returns Array mapowań placeholder → type + path
   */
  static extractPlaceholders(manifest: AssemblyManifest):
PlaceholderMapping[]

  /**
   * Walidacja struktury (bez AI)
   * Sprawdza: kolejność chapters, duplikaty, ścieżki
   */
  static validateStructure(manifest: AssemblyManifest): ValidationError[]

  /**
   * Detekcja typu assetu z rozszerzenia
   */
  private static detectAssetType(path: string): AssetType
}
Usage:

typescript
try {
  const manifest = await ManifestService.parseManifest(manifestFile);
  const placeholders = ManifestService.extractPlaceholders(manifest);
  const errors = ManifestService.validateStructure(manifest);

  if (errors.length > 0) {
    console.error('Validation errors:', errors);
  }
```

```
} catch (e) {
  if (e instanceof AssemblyError) {
    showErrorModal(e.message);
  }
}
```

4.2 AssetProcessor

typescript

```
class AssetProcessor {

  /**
   * Przetwarza uploaded file → ProcessedAsset
   * Wykonuje: sanitization, optimization, checksum
   */
  static async process(
    file: File,
    type: AssetType,
    config: ProcessingConfig
  ): Promise<ProcessedAsset>

  /**
   * HTML: sanitize + extract CSS + wrap sandbox
   */
  private static async processHTML(
    file: File,
    config: ProcessingConfig
  ): Promise<ProcessedAsset>

  /**
   * Image: resize + compress + convert format
   */
  private static async processImage(
    file: File,
    config: ProcessingConfig
  ): Promise<ProcessedAsset>

  /**
   * Audio: validate + optional normalization
   */
  private static async processAudio(
    file: File,
    config: ProcessingConfig
  ): Promise<ProcessedAsset>

  /**
   * Generuj SHA-256 checksum dla integrity
   */
  private static async generateChecksum(blob: Blob): Promise<string>
}
```

Processing Config:

typescript

```
interface ProcessingConfig {
  // Image optimization
  maxWidth?: number;        // Default: 1920
```

```
  maxHeight?: number;       // Default: 1080
  quality?: number;         // 0-1, default: 0.85
  format?: 'image/webp' | 'image/jpeg' | 'image/png';

  // HTML sanitization
  strict?: boolean;         // Default: true
  allowedTags?: string[];
  allowedAttrs?: string[];

  // Audio
  normalize?: boolean;      // Volume normalization
  trimSilence?: boolean;
}
```

4.3 PlaceholderParser

typescript

```
class PlaceholderParser {

  /**
   * Wykryj wszystkie {{PLACEHOLDERS}} w tekście
   * @returns Array unikalnych placeholderów
   */
  static detectInText(text: string): string[]

  /**
   * Zamień placeholdery na rzeczywisty content
   * @param text Tekst źródłowy z placeholderami
   * @param mappings Map placeholder → ProcessedAsset
   * @returns Tekst z wstrzykniętymi assetami
   */
  static async injectContent(
    text: string,
    mappings: Map<string, ProcessedAsset>
  ): Promise<string>

  /**
   * Generuj HTML replacement dla assetu
   * Różne strategie w zależności od typu
   */
  private static async generateReplacement(
    asset: ProcessedAsset
  ): Promise<string>
}
```

Injection Templates:

typescript

```
// WORKCARD_HTML
<section
  data-workcard-id="${asset.id}"
  data-checksum="${asset.checksum}"
  class="workcard-inject"
  style="isolation: isolate;"
>
  ${sanitizedHTML}
</section>
```

```
// AUDIO
<audio controls data-asset-id="${asset.id}" class="audio-inject">
  <source src="${asset.processedURL}" type="audio/mpeg">
</audio>

// IMAGE
<figure data-asset-id="${asset.id}" class="image-inject">
  <img
    src="${asset.processedURL}"
    alt="Asset ${asset.id}"
    loading="lazy"
  />
</figure>
4.4 HTMLSanitizer
typescript
class HTMLSanitizer {

  /**
   * Sanitize HTML przed injection
   * Multi-layer security: pattern check + DOMPurify + sandboxing
   * @throws AssemblyError on security violation
   */
  static sanitize(html: string, strict: boolean = true): string

  /**
   * Wrap HTML w Shadow DOM dla CSS isolation
   * Optional: dla max sandboxing
   */
  static wrapInShadowDOM(html: string, css: string): string

  /**
   * Allowed tags (configurable)
   */
  private static ALLOWED_TAGS: string[]

  /**
   * Forbidden patterns (XSS prevention)
   */
  private static FORBIDDEN_PATTERNS: RegExp[]
}
```

Security Layers:
```
INPUT HTML
    ↓
[1] Pattern Matching (FORBIDDEN_PATTERNS)
    ↓
[2] DOMPurify Sanitization
    ↓
[3] Sandbox Wrapper (isolation: isolate)
    ↓
[4] Optional Shadow DOM
```

```
    ↓
SAFE HTML OUTPUT
```

4.5 AssemblyValidator

typescript

```
class AssemblyValidator {

  /**
   * AI-powered semantic validation
   * WAŻNE: AI nie generuje contentu, tylko waliduje!
   * @param useAI Optional, default: false (use local validation)
   */
  static async validateWithAI(
    manifest: AssemblyManifest,
    mappings: PlaceholderMapping[]
  ): Promise<ValidationResult>

  /**
   * Lokalna walidacja (bez AI, instant)
   * Sprawdza: resolved status, type mismatches, size limits
   */
  static validateLocal(
    mappings: PlaceholderMapping[]
  ): ValidationResult
}
```

Validation Result:

typescript

```
interface ValidationResult {
  status: 'COMPLETE' | 'INCOMPLETE' | 'ERROR';
  total: number;
  mapped: number;
  missing: PlaceholderMapping[];
  orphaned: string[];           // Files bez placeholderów
  errors: ValidationError[];
}

interface ValidationError {
  type: 'MISSING_ASSET' | 'INVALID_PATH' | 'UNSUPPORTED_TYPE';
  placeholder: string;
  message: string;
  severity: 'ERROR' | 'WARNING';
}
```

4.6 BatchMapper

typescript

```
class BatchMapper {

  /**
   * Auto-mapowanie przez naming convention
   * {{WORKCARD_DAY_01}} → workcard_day_01.html
   */
  static autoMapByNaming(
    placeholders: PlaceholderMapping[],
    assets: File[]
  ): Map<string, File>
```

```
  /**
   * Upload całego folderu (preserve structure)
   */
  static async uploadFolder(
    folderInput: HTMLInputElement
  ): Promise<Map<string, File>>

  /**
   * Pattern-based mapping
   * Pattern: "workcard_{day}.html"
   * Matches: {{WORKCARD_DAY_01}} → workcard_01.html
   */
  static mapByPattern(
    placeholders: PlaceholderMapping[],
    assets: File[],
    pattern: string
  ): Map<string, File>
}
```

4.7 ExportEngine

typescript

```
class ExportEngine {

  /**
   * Export z predefined profile
   */
  static async exportWithProfile(
    manifest: AssemblyManifest,
    assembledContent: string,
    assets: ProcessedAsset[],
    profileId: string
  ): Promise<ExportResult>

  /**
   * Export do single HTML file
   * Wszystkie CSS/JS inline, assets as base64 (optional)
   */
  static async exportHTML(
    manifest: AssemblyManifest,
    assembledContent: string,
    assets: ProcessedAsset[]
  ): Promise<ExportResult>

  /**
   * Export do EPUB3 (via worker)
   * Async processing off main thread
   */
  static async exportEPUB(
    manifest: AssemblyManifest,
    assembledContent: string,
    assets: ProcessedAsset[],
    config: EPUBConfig
  ): Promise<ExportResult>
```

```
  /**
   * Export do PDF (via worker)
   * Heavy lifting → dedicated worker
   */
  static async exportPDF(
    manifest: AssemblyManifest,
    assembledContent: string,
    config: PDFConfig
  ): Promise<ExportResult>
}
```

Export Profiles:

typescript

```
const EXPORT_PROFILES = [
  {
    id: 'kindle-standard',
    format: 'epub',
    config: {
      pageSize: 'Kindle',
      colorMode: 'grayscale',
      compression: 'maximum',
      optimizeImages: true
    }
  },
  {
    id: 'print-ready-a4',
    format: 'pdf',
    config: {
      pageSize: 'A4',
      margins: { top: 20, right: 15, bottom: 20, left: 15 },
      colorMode: 'color',
      embedFonts: true
    }
  },
  // ... więcej profili
];
```

4.8 SnapshotManager

typescript

```
class SnapshotManager {

  /**
   * Utwórz snapshot projektu
   * Zapisuje: manifest + mappings + metadata
   */
  static async createSnapshot(
    manifest: AssemblyManifest,
    mappings: Map<string, ProcessedAsset>,
    description?: string
  ): Promise<string>

  /**
   * Załaduj snapshot
   */
  static async loadSnapshot(id: string): Promise<ProjectSnapshot | null>
```

```
  /**
   * Lista wszystkich snapshotów dla projektu
   */
  static async listSnapshots(manifestTitle: string):
Promise<ProjectSnapshot[]>

  /**
   * Auto-save (throttled, max 5 auto-saves)
   */
  static async autoSave(
    manifest: AssemblyManifest,
    mappings: Map<string, ProcessedAsset>
  ): Promise<void>
}
4.9 CollaborationService
typescript
class CollaborationService {

  /**
   * Share projektu (zapisz do shared storage)
   * @returns Share URL
   */
  static async shareProject(
    manifest: AssemblyManifest,
    mappings: Map<string, ProcessedAsset>
  ): Promise<string>

  /**
   * Załaduj shared project
   */
  static async loadSharedProject(projectId: string): Promise<{
    manifest: AssemblyManifest;
    mappings: Map<string, ProcessedAsset>;
  } | null>

  /**
   * Dodaj komentarz do sekcji
   */
  static async addComment(
    projectId: string,
    sectionId: string,
    comment: string,
    author: string
  ): Promise<void>
}
```

---

5. SECURITY & SANDBOXING

5.1 XSS Prevention Strategy

Threat model:
User wstrzykuje złośliwy HTML → XSS → kradzież danych / session hijacking.

Defense layers:
```
USER HTML INPUT
    ↓
[1] FORBIDDEN PATTERN CHECK
    - Regex dla: javascript:, on*, <script>, eval()
    - Reject natychmiast
    ↓
[2] DOMPURIFY SANITIZATION
    - Whitelist allowed tags/attrs
    - Remove wszystkie event handlers
    - Encode dangerous chars
    ↓
[3] SANDBOX WRAPPER
    - CSS isolation: isolation: isolate
    - Contain: layout style paint
    - No script execution context
    ↓
[4] CONTENT SECURITY POLICY (CSP)
    - script-src 'self'
    - object-src 'none'
    - frame-ancestors 'none'
    ↓
SAFE RENDERING
```
5.2 Allowed vs Forbidden
Allowed tags:

typescript
```
['div', 'span', 'p', 'h1-h6', 'ul', 'ol', 'li',
 'table', 'tr', 'td', 'th', 'input', 'textarea',
 'label', 'button', 'select', 'img', 'audio', 'video']
```
Forbidden tags:

typescript
```
['script', 'iframe', 'object', 'embed', 'applet',
 'link', 'style', 'meta', 'base']
```
Forbidden attributes:

typescript
```
['onclick', 'onerror', 'onload', 'onmouseover',
 'onfocus', 'onblur', ...]
```
5.3 Shadow DOM Isolation (Optional)
Dla max security, wrap w Shadow DOM:

html
```
<div class="shadow-host">
  <template shadowrootmode="open">
    <style>
      :host { all: initial; display: block; }
      /* Scoped CSS */
```

```
    </style>
    <!-- Workcard HTML -->
  </template>
</div>
```

Benefits:

CSS nie leakuje na zewnątrz
External CSS nie wpływa na workcard
JS nie może dostać się do parent DOM

6. WORKFLOW GUIDES

6.1 Assembly Mode - Complete Workflow

```
┌──────────────────────────────────────┐
│  1. CREATE MANIFEST                  │
│  - Define metadata                   │
│  - List sections                     │
│  - Map placeholders                  │
└──────────────────────────────────────┘
                │
                ↓
┌──────────────────────────────────────┐
│  2. UPLOAD MANIFEST                  │
│  - ManifestService.parseManifest()   │
│  - Validate schema                   │
│  - Extract placeholders              │
└──────────────────────────────────────┘
                │
                ↓
┌──────────────────────────────────────┐
│  3. MAP ASSETS                       │
│  - Upload files (drag & drop)        │
│  - Auto-map by naming (optional)     │
│  - Batch upload folder (optional)    │
│  - Manual mapping                    │
└──────────────────────────────────────┘
                │
                ↓
┌──────────────────────────────────────┐
│  4. VALIDATE                         │
│  - Incremental validation (live)     │
│  - Local validation (instant)        │
│  - Optional AI validation            │
└──────────────────────────────────────┘
                │
                ↓
┌──────────────────────────────────────┐
│  5. PREVIEW                          │
│  - Live preview assembled content    │
│  - Test interactivity                │
│  - Inspect HTML/CSS                  │
└──────────────────────────────────────┘
                │
```

Continue

20 sty

```
            ↓
┌──────────────────────────────────────┐
│  6. EXPORT                           │
│  - Select export profile             │
│  - Generate final file               │
```

```
│  - Download                                │
└──────────────────────────────────────────┘
```

6.2 Example: 30-Day Workbook

Step 1: Przygotuj strukturę

```
project/
├── manifest.json
├── workcards/
│   ├── day01_breathing.html
│   ├── day02_bodyscan.html
│   └── ... (day03-30)
├── audio/
│   ├── meditation_breathing.mp3
│   ├── meditation_bodyscan.mp3
│   └── ...
├── images/
│   ├── nature_calm.jpg
│   ├── body_awareness.jpg
│   └── ...
└── styles/
    ├── workbook.css
    └── forms.css
```

Step 2: Manifest

```json
{
  "version": "2.0",
  "metadata": { ... },
  "sections": [
    {
      "id": "day-01",
      "chapter": 1,
      "title": "Dzień 1: Świadomy Oddech",
      "placeholders": {
        "{{WORKCARD_01}}": "workcards/day01_breathing.html",
        "{{AUDIO_01}}": "audio/meditation_breathing.mp3",
        "{{IMAGE_01}}": "images/nature_calm.jpg"
      }
    }
    // ... more days
  ]
}
```

Step 3: Workcard HTML (day01_breathing.html)

```html
<div class="workcard">
  <h3>Ćwiczenie: Świadomy Oddech</h3>
  <p>Znajdź wygodną pozycję i zamknij oczy...</p>

  <form class="exercise-form">
```

```
    <label>
      Jak długo ćwiczyłeś/aś? (minuty)
      <input type="number" name="duration" min="1" max="60">
    </label>

    <label>
      Jak się czujesz po ćwiczeniu?
      <textarea name="reflection" rows="4"></textarea>
    </label>

    <button type="submit">Zapisz odpowiedź</button>
  </form>
</div>
```

Step 4: Upload do DTR
- Upload `manifest.json`
- Upload folder `workcards/`
- Upload folder `audio/`
- Upload folder `images/`
- Auto-map by naming

Step 5: Validate & Preview
- Check wszystkie 30 dni mapped
- Preview Day 1 → test form
- Preview Day 15 → check audio player

Step 6: Export
- Profile: "Therapy Workbook" (PDF A4)
- Generate → `30-day-mindfulness-workbook.pdf`
- Download & share

---

7. PERFORMANCE GUIDELINES

7.1 Performance Targets

Operation	Target	Method
Parse manifest	<100ms	Sync JSON.parse
Validate schema	<200ms	AJV validator
Process 1 image	<500ms	OffscreenCanvas
Process 1 HTML	<300ms	DOMPurify
Map 50 assets	<2s	Async parallel
Validate (local)	<500ms	Sync checks
Validate (AI)	<5s	Single API call
Inject placeholders	<3s	String replacement batch
Export HTML	<1s	Template + bundle
Export EPUB	<5s	Worker thread
Export PDF	<15s	Worker thread

7.2 Optimization Strategies

1. Lazy Loading Assets
```typescript
// Don't load all assets upfront
const asset = mappings.get(placeholder);
if (!asset.loaded) {
  await loadAsset(asset); // Load on-demand
}
```

2. Batch Processing
```typescript
// Process images in parallel
await Promise.all(
  images.map(img => AssetProcessor.process(img, 'raster', config))
);
```

3. Worker Threads dla Heavy Operations
```typescript
// PDF generation off main thread
const worker = new Worker('/workers/pdf.worker.js');
worker.postMessage({ content, config });
```

4. Incremental Rendering
```typescript
// Don't wait for all sections, render progressively
sections.forEach((section, i) => {
  setTimeout(() => renderSection(section), i * 100);
});
```

5. Memoization
```typescript
const processedCache = new Map<string, ProcessedAsset>();

async function getProcessed(file: File): Promise<ProcessedAsset> {
  const key = `${file.name}_${file.size}`;
  if (processedCache.has(key)) {
    return processedCache.get(key)!;
  }
  const processed = await AssetProcessor.process(file);
  processedCache.set(key, processed);
  return processed;
}
```

7.3 Memory Management

Problem: 50+ high-res images = memory leak.

Solution:
```typescript
// Revoke object URLs after use
```

```
function cleanup(assets: ProcessedAsset[]) {
  assets.forEach(asset => {
    if (asset.processedURL.startsWith('blob:')) {
      URL.revokeObjectURL(asset.processedURL);
    }
  });
}

// Call after export
await ExportEngine.exportPDF(...);
cleanup(processedAssets);
```

---

8. TESTING STRATEGY

8.1 Unit Tests (Jest)
```typescript
// manifestService.test.ts
describe('ManifestService', () => {

  it('should parse valid manifest', async () => {
    const file = new File([validManifestJSON], 'manifest.json');
    const manifest = await ManifestService.parseManifest(file);
    expect(manifest.version).toBe('2.0');
  });

  it('should reject invalid schema', async () => {
    const file = new File([invalidJSON], 'manifest.json');
    await expect(ManifestService.parseManifest(file)).rejects.toThrow();
  });

  it('should extract placeholders correctly', () => {
    const placeholders = ManifestService.extractPlaceholders(manifest);
    expect(placeholders).toHaveLength(60); // 30 days * 2 placeholders
  });
});
```

8.2 Integration Tests
```typescript
// assembly.integration.test.ts
describe('Assembly Pipeline', () => {

  it('should complete full assembly workflow', async () => {
    const manifest = await ManifestService.parseManifest(manifestFile);
    const placeholders = ManifestService.extractPlaceholders(manifest);

    // Map assets
    const mappings = await Promise.all(
      placeholders.map(async p => {
        const file = assetFiles.find(f =>
f.name.includes(p.placeholder));
```

```
        const processed = await AssetProcessor.process(file!, p.type,
{});
        return [p.placeholder, processed];
      })
    );

    // Validate
    const validation = AssemblyValidator.validateLocal(placeholders);
    expect(validation.status).toBe('COMPLETE');

    // Inject
    const assembled = await PlaceholderParser.injectContent(
      manuscriptText,
      new Map(mappings)
    );

    expect(assembled).not.toContain('{{');
  });
});
```

8.3 E2E Tests (Playwright)

```typescript
// assembly.e2e.test.ts
test('User uploads manifest and exports HTML', async ({ page }) => {
  await page.goto('/');

  // Select Assembly Mode
  await page.click('[data-testid="mode-assembly"]');

  // Upload manifest
  await page.setInputFiles('input[type="file"]', 'test-
data/manifest.json');
  await page.waitForSelector('[data-testid="placeholders-list"]');

  // Upload assets folder
  await page.setInputFiles('input[webkitdirectory]', 'test-
data/assets/');

  // Auto-map
  await page.click('[data-testid="auto-map-button"]');
  await page.waitForSelector('[data-testid="validation-complete"]');

  // Export
  await page.click('[data-testid="export-html"]');
  const download = await page.waitForEvent('download');
  expect(download.suggestedFilename()).toBe('assembled-ebook.html');
});
```

---

9. DEPLOYMENT

9.1 Build Configuration
```json
// package.json
{
  "scripts": {
    "dev": "vite",
    "build": "tsc && vite build",
    "preview": "vite preview",
    "test": "jest",
    "test:e2e": "playwright test",
    "lint": "eslint src/",
    "type-check": "tsc --noEmit"
  },
  "dependencies": {
    "react": "^19.0.0",
    "react-dom": "^19.0.0",
    "mammoth": "^1.6.0",
    "ajv": "^8.12.0",
    "dompurify": "^3.0.0"
  },
  "devDependencies": {
    "@types/react": "^19.0.0",
    "vite": "^5.0.0",
    "typescript": "^5.3.0",
    "jest": "^29.0.0",
    "playwright": "^1.40.0"
  }
}
```

9.2 Environment Variables
```bash
# .env.production
VITE_AI_API_KEY=your_key_here
VITE_STORAGE_ENDPOINT=https://storage.dtr.studio
VITE_MAX_FILE_SIZE=10485760  # 10MB
VITE_ENABLE_COLLABORATION=true
```

9.3 Claude Artifacts Deployment

Requirements:
- Single HTML artifact (all deps via CDN)
- No localStorage/sessionStorage
- Use Claude storage API for persistence

Entry point:
```html
<!-- index.html -->
<!DOCTYPE html>
<html>
<head>
  <script src="https://cdn.tailwindcss.com"></script>
  <script type="importmap">
```

```
  {
    "imports": {
      "react": "https://esm.sh/react@19",
      "react-dom": "https://esm.sh/react-dom@19",
      "mammoth": "https://esm.sh/mammoth@1.6.0"
    }
  }
  </script>
</head>
<body>
  <div id="root"></div>
  <script type="module" src="/index.tsx"></script>
</body>
</html>
```

---

10. TROUBLESHOOTING

10.1 Common Issues

Problem: Manifest validation fails
Error: Invalid manifest: data.sections[0].chapter should be >= 1

Solution: Check chapter numbering starts at 1, is sequential.

Problem: Asset not mapping
Warning: Placeholder {{WORKCARD_01}} not resolved

Solution:
1. Check filename matches manifest path exactly
2. Use batch auto-map
3. Verify file uploaded successfully

Problem: HTML injection fails / blank output
Error: DOMPurify removed all content

Solution:
1. Check HTML doesn't contain `<script>` tags
2. Remove inline event handlers (`onclick=...`)
3. Use allowed tags only

Problem: PDF export hangs
Worker timeout after 30s

Solution:
1. Reduce image count/size

2. Split into multiple PDFs
3. Use "draft" quality profile

Problem: Memory leak on large projects
Out of memory error after processing 100+ images

Solution:
```typescript
// Revoke blob URLs periodically
setInterval(() => {
  processedAssets.forEach(a => {
    if (!a.inUse) URL.revokeObjectURL(a.processedURL);
  });
}, 60000); // Every minute
```

10.2 Debug Mode

Enable verbose logging:
```typescript
// Add to App.tsx
if (import.meta.env.DEV) {
  window.DTR_DEBUG = true;
}

// In services
if (window.DTR_DEBUG) {
  console.log('[ManifestService] Parsing:', file.name);
  console.log('[AssetProcessor] Processing:', asset.type);
}
```

KONIEC DOKUMENTACJI TECHNICZNEJ

Next steps:
- Implementacja kodu
- UI/UX mockupy
- Example projects
- Video tutorials

Pytania? Issues?
→ GitHub Issues: `https://github.com/hardbanrecords/dtr-studio/issues`

Version: 2.0.0
Last Updated: 2025-01-20
Authors: DTR Engineering Team

Continuenapisz mi p[rompt dla ai aby na podstawoe dokumentow stworzyl pelno wymiarowa app

21 sty
PROMPT DLA AI - STWORZENIE DTR STUDIO ENGINE v2.0
Jesteś ekspertem React/TypeScript developer. Twoim zadaniem jest stworzenie kompletnej,
produkcyjnej aplikacji DTR STUDIO ENGINE v2.0 na podstawie poniższej dokumentacji.

═══
KONTEKST I DOKUMENTACJA
═══

Masz dostęp do następujących dokumentów:
1. App.tsx (obecna implementacja - jako punkt odniesienia)
2. Raport z analizy obecnej aplikacji
3. Dokument dopracowania architektury
4. Pełna dokumentacja techniczna v2.0

Przeczytaj je dokładnie przed rozpoczęciem implementacji.

═══
KLUCZOWE WYMAGANIA PROJEKTU
═══

1. FILOZOFIA PRODUKTU

DTR Studio Engine to silnik SKŁADU i INTEGRACJI ebooków interaktywnych.

KRYTYCZNE ZASADY:
✓ AI NIE MODYFIKUJE TREŚCI AUTORSKIEJ (zero ingerencji)
✓ Manifest-driven assembly (deterministyczny workflow)
✓ Security-first (sandboxed HTML injection)
✓ Dual-mode architecture (Assembly + Studio)

2. DUAL-MODE ARCHITECTURE

ASSEMBLY MODE (priorytet):
- Input: manifest.json + assets (HTML/audio/images)
- Proces: placeholders → assets mapping → validation → export
- AI: TYLKO opcjonalna walidacja (nie tworzy contentu!)
- Use case: workbooki terapeutyczne, kursy, ebooki interaktywne

STUDIO MODE (obecny tryb):
- Input: manuscript (DOCX/MD/TXT)
- Proces: AI layout orchestration → Visual DNA → typography
- Use case: publikacje wizualne, DTP, print-ready

3. KLUCZOWE FUNKCJE DO ZAIMPLEMENTOWANIA

ASSEMBLY MODE - CORE FEATURES:

1. **Mode Selector** (ekran startowy)
   - Wybór: Assembly vs Studio
   - Opis każdego trybu
   - Wizualne rozróżnienie

2. **Manifest Upload & Validation**
   - Drag & drop manifest.json
   - JSON Schema validation (AJV)
   - Wyświetlenie struktury projektu
   - Ekstrakcja placeholderów

3. **Asset Mapping Interface**
   - Drag & drop dla pojedynczych plików
   - Upload całego folderu (preserve structure)
   - Auto-mapping by naming convention
   - Pattern-based mapping
   - Manual mapping (drag placeholder → file)
   - Live visual feedback (green = mapped, red = missing)

4. **Incremental Validation**
   - Real-time validation podczas mappingu
   - Progress bar (X/Y placeholders mapped)
   - Error/warning display
   - Type mismatch detection
   - File size warnings

5. **Placeholder Tree View**
   - Hierarchiczna struktura sekcji
   - Status każdego placeholder (✓ mapped / ✗ missing)
   - Click → jump to mapping panel
   - Search/filter placeholders

6. **Asset Processing Pipeline**
   - HTML: sanitize (DOMPurify) + extract CSS + sandbox wrapper
   - Images: resize + compress + WebP conversion
   - Audio: validation + metadata extraction
   - Checksum generation (SHA-256)

7. **Live Preview**
   - Split view: structure tree (left) + rendered preview (right)
   - Test interactivity (forms, buttons)
   - Inspect mode (view HTML/CSS)
   - Responsive preview (mobile/tablet/desktop)

8. **Export Engine**
   - Export profiles (Kindle, Apple Books, Print PDF, Web HTML)
   - Profile configuration UI
   - Worker-based PDF/EPUB generation
   - Progress indicator
   - Download final file

9. **Advanced Features**
   - Undo/Redo (Command pattern)
   - Project snapshots (version control)
   - Auto-save (every 2 min, keep last 5)
   - Collaboration (share project via URL)
   - Batch operations UI

STUDIO MODE - ZACHOWAJ OBECNE FUNKCJE:

- Visual DNA Matrix (80+ templates)
- AI Layout Orchestration (Gemini)
- Asset Configuration
- Preflight Proofing
- Typography engine

═══
═══════

STRUKTURA PROJEKTU DO UTWORZENIA

═══
═══════

```
src/
├── modes/
│   ├── assembly/
│   │   ├── AssemblyWorkflow.tsx          # Main workflow orchestrator
│   │   ├── ModeSelector.tsx              # Entry point (Assembly vs
Studio)
│   │   ├── ManifestUpload.tsx            # Step 1: Upload manifest
│   │   ├── AssetMapper.tsx               # Step 2: Map assets to
placeholders
│   │   ├── ValidationDashboard.tsx       # Step 3: Validation results
│   │   ├── AssemblyPreview.tsx           # Step 4: Live preview
│   │   └── ExportPanel.tsx               # Step 5: Export with profiles
│   │
│   └── studio/
│       ├── StudioWorkflow.tsx            # Existing workflow (refactor
from App.tsx)
│       ├── ManuscriptIngest.tsx
│       ├── AssetConfiguration.tsx
│       └── PreflightProofing.tsx
│
├── services/
│   ├── assembly/
│   │   ├── manifestService.ts            # Parse & validate manifest
│   │   ├── placeholderParser.ts          # Detect & inject placeholders
│   │   ├── assetProcessor.ts             # Process files
(HTML/images/audio)
│   │   ├── htmlSanitizer.ts              # Security: DOMPurify +
sandboxing
│   │   ├── assemblyValidator.ts          # Local + AI validation
│   │   ├── batchMapper.ts                # Auto-map, folder upload,
patterns
│   │   ├── exportEngine.ts               # HTML/EPUB/PDF export
```

```
│   │   ├── exportProfiles.ts            # Predefined export
configurations
│   │   ├── snapshotManager.ts           # Version control / snapshots
│   │   ├── incrementalValidator.ts      # Real-time validation stream
│   │   └── collaborationService.ts      # Share projects
│   │
│   ├── studio/
│   │   ├── geminiService.ts             # Existing AI service
│   │   ├── metadataService.ts
│   │   └── qrService.ts
│   │
│   └── shared/
│       ├── storageService.ts            # Claude storage API wrapper
│       ├── commandStack.ts              # Undo/Redo implementation
│       └── workerPool.ts                # Web Workers manager
│
├── workers/
│   ├── pdf.worker.ts                    # PDF generation (heavy)
│   ├── epub.worker.ts                   # EPUB generation
│   └── validation.worker.ts             # Optional: AI validation off-
thread
│
├── types/
│   ├── assembly.types.ts                # Assembly mode types
│   ├── studio.types.ts                  # Studio mode types (from
existing)
│   └── shared.types.ts                  # Common types
│
├── components/
│   ├── shared/
│   │   ├── Sidebar.tsx                  # Navigation sidebar
│   │   ├── StepIndicator.tsx            # Progress indicator
│   │   └── Header.tsx                   # Top header
│   │
│   └── assembly/
│       ├── PlaceholderTree.tsx          # Hierarchical placeholder list
│       ├── AssetMappingPanel.tsx        # Drag & drop asset mapping
│       ├── ValidationReport.tsx         # Validation errors/warnings
│       ├── BatchOperations.tsx          # Auto-map, folder upload UI
│       ├── SnapshotPanel.tsx            # Version history UI
│       └── CollaborationPanel.tsx       # Share project UI
│
├── schemas/
│   └── manifest.schema.json             # JSON Schema for manifest
validation
│
├── constants/
│   ├── exportProfiles.ts                # Export profile definitions
│   └── templates.ts                     # Visual DNA templates
(existing)
│
└── App.tsx                              # Main router (mode selection)
```

═══
═══════
KRYTYCZNE IMPLEMENTACJE
═══
═══════

A. MANIFEST SERVICE (manifestService.ts)

Implementuj zgodnie z dokumentacją:
```typescript
import Ajv from 'ajv';
import manifestSchema from '../schemas/manifest.schema.json';

export class ManifestService {
  static async parseManifest(file: File): Promise<AssemblyManifest> {
    // 1. Read file
    // 2. Parse JSON
    // 3. Validate against schema (AJV)
    // 4. Return typed manifest
  }

  static extractPlaceholders(manifest: AssemblyManifest):
PlaceholderMapping[] {
    // 1. Iterate sections
    // 2. Extract all {{PLACEHOLDERS}}
    // 3. Detect asset type from file extension
    // 4. Return array of mappings
  }

  static validateStructure(manifest: AssemblyManifest): ValidationError[]
{
    // 1. Check chapter ordering
    // 2. Detect duplicate placeholders
    // 3. Validate paths
    // 4. Return errors
  }
}
```

B. HTML SANITIZER (htmlSanitizer.ts)

KRYTYCZNE DLA BEZPIECZEŃSTWA:
```typescript
import DOMPurify from 'dompurify';

export class HTMLSanitizer {
  private static FORBIDDEN_PATTERNS = [
    /javascript:/gi,
    /on\w+=/gi,
    /<script/gi,
    /<iframe/gi,
    /eval\(/gi
  ];
```

```
  static sanitize(html: string, strict: boolean = true): string {
    // LAYER 1: Pattern check
    for (const pattern of this.FORBIDDEN_PATTERNS) {
      if (pattern.test(html)) {
        throw new AssemblyError('SECURITY_VIOLATION', 'ERROR',
          `Forbidden pattern: ${pattern.source}`);
      }
    }

    // LAYER 2: DOMPurify
    const clean = DOMPurify.sanitize(html, {
      ALLOWED_TAGS: this.ALLOWED_TAGS,
      ALLOWED_ATTR: this.ALLOWED_ATTRS,
      FORBID_TAGS: ['script', 'iframe', 'object'],
      FORBID_ATTR: ['onerror', 'onload', 'onclick']
    });

    // LAYER 3: Sandbox wrapper
    return `
      <div class="workcard-sandbox" style="isolation: isolate;">
        ${clean}
      </div>
    `;
  }
}
```

C. ASSET PROCESSOR (assetProcessor.ts)

```typescript
export class AssetProcessor {
  static async process(
    file: File,
    type: AssetType,
    config: ProcessingConfig
  ): Promise<ProcessedAsset> {

    switch (type) {
      case 'workcard-html':
        return this.processHTML(file, config);
      case 'raster':
        return this.processImage(file, config);
      case 'audio':
        return this.processAudio(file, config);
    }
  }

  private static async processImage(file: File, config: ProcessingConfig)
{
    // 1. Create ImageBitmap
    // 2. Resize via OffscreenCanvas
    // 3. Convert to WebP
    // 4. Generate checksum
    // 5. Return ProcessedAsset
  }
```

```
  private static async generateChecksum(blob: Blob): Promise<string> {
    const buffer = await blob.arrayBuffer();
    const hashBuffer = await crypto.subtle.digest('SHA-256', buffer);
    return Array.from(new Uint8Array(hashBuffer))
      .map(b => b.toString(16).padStart(2, '0'))
      .join('');
  }
}
```

D. EXPORT ENGINE (exportEngine.ts)

```typescript
export class ExportEngine {
  static async exportHTML(
    manifest: AssemblyManifest,
    assembledContent: string,
    assets: ProcessedAsset[]
  ): Promise<ExportResult> {

    const html = `
<!DOCTYPE html>
<html lang="${manifest.metadata.language}">
<head>
  <meta charset="UTF-8">
  <title>${manifest.metadata.title}</title>
  <style>${await this.bundleCSS(manifest.assets.css)}</style>
</head>
<body>
  ${assembledContent}
  <script>${await this.bundleScripts(manifest.assets.scripts)}</script>
</body>
</html>
    `;

    const blob = new Blob([html], { type: 'text/html' });
    const url = URL.createObjectURL(blob);

    return {
      success: true,
      format: 'html',
      size: blob.size,
      downloadURL: url,
      metadata: { assets: assets.length, generatedAt: new
Date().toISOString() }
    };
  }

  static async exportPDF(...) {
    // Use Web Worker for heavy processing
    const worker = new Worker(new URL('../workers/pdf.worker.ts',
import.meta.url));
    return new Promise((resolve, reject) => {
      worker.postMessage({ manifest, content, config });
```

```
      worker.onmessage = (e) => {
        if (e.data.success) resolve(e.data);
        else reject(e.data.error);
        worker.terminate();
      };
    });
  }
}
```

═══
═══════
UI/UX REQUIREMENTS
═══
═══════

DESIGN SYSTEM (zachowaj istniejący styl):

- Kolory: slate-900 (dark), amber-500 (accent), white
- Typography: Inter (UI), Crimson Pro (content)
- Rounded corners: 2xl-4xl dla main elements
- Shadows: xl-2xl
- Animations: smooth transitions (duration-300-500)

MODE SELECTOR SCREEN:

```tsx
<div className="h-screen flex items-center justify-center bg-[#020617]">
  <div className="max-w-7xl mx-auto px-20">
    <header className="text-center mb-20">
      <h1 className="text-7xl font-black uppercase tracking-tighter text-
white mb-6">
        DTR Studio Engine
      </h1>
      <p className="text-slate-500 text-sm uppercase tracking-[0.4em]">
        Choose Your Workflow
      </p>
    </header>

    <div className="grid grid-cols-2 gap-20">
      {/* ASSEMBLY MODE CARD */}
      <button className="group p-20 bg-slate-900 rounded-[4rem] border-2
border-white/5
                         hover:border-amber-500/50 transition-all">
        <Command size={80} className="text-amber-500 mb-10" />
        <h2 className="text-4xl font-black uppercase text-white mb-6">
          Assembly Mode
        </h2>
        <p className="text-slate-400 text-sm leading-relaxed mb-10">
          Manifest-driven ebook assembly. Perfect for workbooks,
          courses, and interactive publications.
        </p>
        <div className="flex gap-3 text-xs text-slate-600 uppercase
tracking-widest">
```

```
          <span className="px-4 py-2 bg-slate-800 rounded-xl">Zero AI
Content</span>
          <span className="px-4 py-2 bg-slate-800 rounded-
xl">Deterministic</span>
        </div>
      </button>

      {/* STUDIO MODE CARD */}
      <button className="group p-20 bg-slate-900 rounded-[4rem] border-2
border-white/5
                         hover:border-amber-500/50 transition-all">
        <Palette size={80} className="text-amber-500 mb-10" />
        <h2 className="text-4xl font-black uppercase text-white mb-6">
          Studio Mode
        </h2>
        <p className="text-slate-400 text-sm leading-relaxed mb-10">
          AI-powered layout orchestration. Perfect for novels,
          portfolios, and print-ready publications.
        </p>
        <div className="flex gap-3 text-xs text-slate-600 uppercase
tracking-widest">
          <span className="px-4 py-2 bg-slate-800 rounded-xl">Visual
DNA</span>
          <span className="px-4 py-2 bg-slate-800 rounded-xl">Typography
AI</span>
        </div>
      </button>
    </div>
  </div>
</div>
```

ASSET MAPPER INTERFACE:

```tsx
<div className="grid grid-cols-12 gap-10">
  {/* LEFT: Placeholder Tree */}
  <div className="col-span-5 bg-white rounded-[3rem] p-10">
    <h3 className="text-xs font-black uppercase mb-8">Placeholders</h3>

    <div className="space-y-4">
      {sections.map(section => (
        <div key={section.id}>
          <h4 className="font-black text-sm mb-3">{section.title}</h4>
          {Object.keys(section.placeholders).map(ph => (
            <div
              className={`p-4 rounded-xl mb-2 ${
                mappings.get(ph)?.resolved
                  ? 'bg-green-50 border-2 border-green-500'
                  : 'bg-red-50 border-2 border-red-300 border-dashed'
              }`}
            >
              <code className="text-xs font-bold">{ph}</code>
              {mappings.get(ph)?.resolved && (
```

```
                <CheckCircle2 size={16} className="inline ml-2 text-
green-500" />
              )}
            </div>
          ))}
        </div>
      ))}
    </div>
  </div>

  {/* RIGHT: Asset Upload & Mapping */}
  <div className="col-span-7 space-y-8">
    {/* Drag & Drop Zone */}
    <div
      className="border-4 border-dashed border-slate-300 rounded-[3rem]
                 p-20 text-center hover:border-amber-500 transition-all"
      onDrop={handleDrop}
      onDragOver={handleDragOver}
    >
      <Upload size={60} className="mx-auto mb-6 text-slate-400" />
      <p className="text-sm font-black uppercase tracking-widest text-
slate-600">
        Drop assets here or click to browse
      </p>
    </div>

    {/* Batch Operations */}
    <div className="bg-slate-900 p-8 rounded-3xl">
      <h4 className="text-xs font-black uppercase text-amber-500 mb-6">
        Batch Operations
      </h4>
      <div className="space-y-3">
        <button className="w-full py-4 bg-amber-500 rounded-xl">
          <Zap size={16} className="inline mr-2" />
          Auto-Map by Naming
        </button>
        <button className="w-full py-4 bg-slate-800 rounded-xl">
          <FolderPlus size={16} className="inline mr-2" />
          Upload Folder
        </button>
      </div>
    </div>
  </div>
</div>
```

═══
═══════

STORAGE & STATE MANAGEMENT

═══
═══════

STORAGE STRATEGY:

NIGDY NIE UŻYWAJ localStorage/sessionStorage (nie działa w Claude Artifacts)

ZAMIAST TEGO:
```typescript
// services/shared/storageService.ts

export class StorageService {

  // Save manifest
  static async saveManifest(projectId: string, manifest:
AssemblyManifest) {
    await window.storage.set(
      `manifest:${projectId}`,
      JSON.stringify(manifest),
      false // personal data
    );
  }

  // Save asset mappings (split into chunks if > 5MB)
  static async saveMappings(projectId: string, mappings: Map<string,
ProcessedAsset>) {
    const entries = Array.from(mappings.entries());
    const serialized = JSON.stringify(entries);

    if (serialized.length > 5_000_000) {
      // Split into chunks
      const chunkSize = 4_000_000;
      for (let i = 0; i < serialized.length; i += chunkSize) {
        const chunk = serialized.slice(i, i + chunkSize);
        await window.storage.set(
          `mappings:${projectId}:chunk${i / chunkSize}`,
          chunk,
          false
        );
      }
    } else {
      await window.storage.set(
        `mappings:${projectId}`,
        serialized,
        false
      );
    }
  }

  // Load project
  static async loadProject(projectId: string): Promise<{
    manifest: AssemblyManifest;
    mappings: Map<string, ProcessedAsset>;
  } | null> {
    try {
      const manifestResult = await
window.storage.get(`manifest:${projectId}`);
      if (!manifestResult) return null;
```

```
      // Load mappings (check for chunks)
      let mappingsData = '';
      try {
        const direct = await window.storage.get(`mappings:${projectId}`);
        mappingsData = direct ? direct.value : '';
      } catch {
        // Load chunks
        let chunkIndex = 0;
        while (true) {
          try {
            const chunk = await
window.storage.get(`mappings:${projectId}:chunk${chunkIndex}`);
            if (!chunk) break;
            mappingsData += chunk.value;
            chunkIndex++;
          } catch {
            break;
          }
        }
      }

      return {
        manifest: JSON.parse(manifestResult.value),
        mappings: new Map(JSON.parse(mappingsData))
      };
    } catch {
      return null;
    }
  }
}
```

STATE MANAGEMENT:

Use React hooks (useState, useReducer, useContext):
```typescript
// Assembly workflow state
const [step, setStep] = useState<AssemblyStep>('MANIFEST_UPLOAD');
const [manifest, setManifest] = useState<AssemblyManifest | null>(null);
const [placeholders, setPlaceholders] =
useState<PlaceholderMapping[]>([]);
const [mappings, setMappings] = useState<Map<string, ProcessedAsset>>(new
Map());
const [validation, setValidation] = useState<ValidationResult |
null>(null);
const [isProcessing, setIsProcessing] = useState(false);

// Command stack for undo/redo
const commandStack = useRef(new CommandStack());
```

═══
═══

IMPLEMENTATION INSTRUCTIONS

STEP-BY-STEP PLAN:

1. **Stwórz strukturę folderów** (jak w sekcji STRUKTURA PROJEKTU)

2. **Zaimplementuj types** (assembly.types.ts, shared.types.ts)

3. **Stwórz core services** (w kolejności):
   - manifestService.ts
   - htmlSanitizer.ts
   - assetProcessor.ts
   - placeholderParser.ts
   - assemblyValidator.ts
   - exportEngine.ts

4. **Zbuduj UI components** (Assembly Mode):
   - ModeSelector.tsx (ekran startowy)
   - ManifestUpload.tsx
   - AssetMapper.tsx
   - PlaceholderTree.tsx
   - ValidationDashboard.tsx
   - ExportPanel.tsx

5. **Zintegruj workflows**:
   - AssemblyWorkflow.tsx (orchestrator)
   - Refactor StudioWorkflow.tsx (z obecnego App.tsx)

6. **Dodaj advanced features**:
   - CommandStack (undo/redo)
   - SnapshotManager
   - BatchMapper
   - IncrementalValidator

7. **Stwórz workers**:
   - pdf.worker.ts
   - epub.worker.ts

8. **Połącz w App.tsx** (mode router)

CRITICAL REMINDERS

✗ NIGDY nie używaj localStorage/sessionStorage
✓ ZAWSZE używaj window.storage API

✗ NIGDY nie pozwalaj AI modyfikować treści autorskiej w Assembly Mode
✓ ZAWSZE traktuj manifest jako single source of truth

✗ NIGDY nie wstrzykuj niesanityzowanego HTML
✓ ZAWSZE przepuść przez HTMLSanitizer

✗ NIGDY nie blokuj main thread przy PDF/EPUB export
✓ ZAWSZE używaj Web Workers dla heavy operations

✗ NIGDY nie ładuj wszystkich assets na raz
✓ ZAWSZE używaj lazy loading i batch processing

═══
═══════
OUTPUT FORMAT
═══
═══════

Stwórz kompletny, gotowy do uruchomienia artifact z:

1. Wszystkimi plikami źródłowymi (ts/tsx)
2. Pełną implementacją wszystkich serwisów
3. Wszystkimi komponentami UI
4. Manifest schema (JSON)
5. Example manifest.json
6. Komentarzami wyjaśniającymi kluczowe decyzje

Kod musi być:
- Production-ready (bez TODO, FIXME)
- Fully typed (TypeScript strict mode)
- Commented (JSDoc dla public APIs)
- Secure (wszystkie inputs sanitized)
- Performant (lazy loading, workers, memoization)

═══
═══════

START IMPLEMENTATION NOW.

Rozpocznij od stworzenia:
1. Kompletnej struktury typów (types/)
2. Core services (services/assembly/)
3. Main components (modes/assembly/)
4. Final integration (App.tsx)

Generuj kod iteracyjnie, sekcja po sekcji, prosząc o feedback po każdej większej części.
Czy chcesz, żebym teraz użył tego promptu i zacząłem implementację, czy jeszcze coś dopracujemy w samym prompcie?

tak

24 sty
DTR STUDIO ENGINE v2.0 - IMPLEMENTACJA

Zaczynam implementację. Podzielę to na sekcje, abyś mógł śledzić postęp i dawać feedback.

CZĘŚĆ 1: TYPE SYSTEM
DTR Studio Engine v2.0 - Assembly First
Interactive artifact
An error occurred while trying to run the generated artifact.
Unexpected token, expected ";" (752:31)

Can you fix this error in dtr_engine_v2?

24 sty
Kontynuuję implementację - dodaję resztę komponentów i główny App:

DTR Studio Engine v2.0 - Assembly First
Interactive artifact · Version 2
✅ IMPLEMENTACJA ZAKOŃCZONA!

Stworzyłem pełnowymiarową aplikację DTR Studio Engine v2.0 z następującymi funkcjami:

🎯 Zaimplementowane Features:
1. Dual-Mode Architecture
Mode Selector (ekran startowy)
Assembly Mode (w pełni funkcjonalny)
Studio Mode (placeholder na przyszłość)
2. Assembly Mode - Complete Workflow
✅ Manifest Upload - drag & drop, JSON validation, error handling
✅ Asset Mapping - automatyczne i manualne mapowanie
✅ Batch Operations - auto-map by naming convention
✅ Real-time Progress - live feedback podczas mappingu
✅ Export Panel - 4 profile eksportu (HTML/EPUB/PDF)
3. Core Services
✅ ManifestService - parse, validate, extract placeholders
✅ AssetProcessor - process files, generate checksums
✅ BatchMapper - inteligentne auto-mapowanie
✅ StorageService - persist do Claude storage API
4. Security
✅ JSON schema validation
✅ File type detection
✅ Checksum generation (SHA-256)
✅ Error handling na każdym poziomie
5. UX/UI
✅ Collapsible sidebar
✅ Step-based navigation
✅ Progress indicators
✅ Drag & drop everywhere
✅ Responsive design
✅ Smooth animations

🎁 Przykładowy Manifest (do testowania):
json
```
{
  "version": "2.0",
  "metadata": {
    "title": "30-Day Mindfulness Workbook",
    "author": "Dr. Example",
    "language": "en"
  },
  "sections": [
    {
      "id": "day-01",
      "chapter": 1,
      "title": "Day 1: Conscious Breathing",
      "placeholders": {
        "{{WORKCARD_01}}": "workcards/day01.html",
        "{{AUDIO_01}}": "audio/meditation01.mp3"
      }
    },
    {
      "id": "day-02",
      "chapter": 2,
      "title": "Day 2: Body Scan",
      "placeholders": {
        "{{WORKCARD_02}}": "workcards/day02.html",
        "{{AUDIO_02}}": "audio/meditation02.mp3"
      }
    }
  ]
}
```
🚀 Następne Kroki (opcjonalne rozszerzenia):
HTML Sanitizer - dodać DOMPurify dla bezpieczeństwa
PDF Worker - off-thread PDF generation
Validation Dashboard - szczegółowe błędy i warningi
Snapshot Manager - version control
Collaboration - share projects via URL
Chcesz, żebym dodał którąś z tych funkcji, czy aplikacja jest już gotowa do użycia?

wlasnie niw bo calkowicie nie rozumiem ci i jak ma ina dziaklac brakuje pezejrzysrgo dasboardu workspace

24 sty
Masz rację - aplikacja jest zbyt techniczna i niejasna. Stworzę teraz PEŁNY, PRZEJRZYSTY WORKSPACE z intuicyjnym dashboardem.

DTR Studio - Ebook Assembly Workspace
Interactive artifact · Version 3
✅ GOTOWE! Teraz aplikacja jest PRZEJRZYSTA i INTUICYJNA
🎯 Co masz teraz:
1. DASHBOARD (główny widok)
✅ Edycja tytułu i autora projektu
✅ Statystyki (liczba rozdziałów)

✅ Lista wszystkich rozdziałów z akcjami
✅ Przycisk "Dodaj Rozdział"
✅ Przyciski "Eksportuj HTML" i "Podgląd"
2. EDYTOR ROZDZIAŁU
✅ Edycja tytułu rozdziału
✅ Duże pole tekstowe na treść
✅ Panel załączników (obrazy, audio, karty HTML)
✅ Drag & drop plików
✅ Przycisk "Zapisz Zmiany"
3. PODGLĄD
✅ Profesjonalny podgląd jak wygląda ebook
✅ Wszystkie rozdziały sformatowane
✅ Wyświetlanie załączników (obrazy, audio)
4. EKSPORT
✅ Jeden klik → pobierasz gotowy plik HTML
✅ Można otworzyć w przeglądarce
✅ Gotowy do konwersji na PDF/EPUB
📖 JAK UŻYWAĆ:
Wpisz tytuł i autora na górze
Kliknij "Dodaj Rozdział"
Wpisz tytuł rozdziału (np. "Dzień 1: Wprowadzenie")
Napisz treść w dużym polu tekstowym
Dodaj załączniki (obrazy, audio, HTML) - kliknij "+" w prawym panelu
Zapisz - wrócisz do dashboardu
Powtórz dla kolejnych rozdziałów
Kliknij "Podgląd" aby zobaczyć jak wygląda
Kliknij "Eksportuj HTML" aby pobrać gotowy plik
🎨 DESIGN:
Ciemny interfejs (mniej męczący dla oczu)
Duże, czytelne przyciski
Kolory: pomarańczowy (akcje), niebieski (podgląd), zielony (zapisz)
Wszystko w jednym widoku - zero zagubienia
Teraz jest jasne jak działa? Chcesz coś zmienić/ulepszyć?

TEMPLATE_GUIDE.md
238 lines

md

Wireframe'y i Mockupy - Interaktywny Szablon Kursu Premium.md
358 lines

md

🏆 Struktura Szablonu Kursu - Wersja OUTSTANDING.md
701 lines

md

WEBBOOK_TEMPLATE.html
1,207 lines

html

WEBBOOK_TEMPLATE_V2.html
1,448 lines

html

🎓 HardbanRecordsLab - Webbook Studio.md
333 lines

md

📖 INSTRUKCJA SZABLONU WEBBOOKA.md
440 lines

md

a teraz dostosuj DTR STUDIO do generowania tworzenai takich webbookow co w za;lacznikach

24 sty
Dostosuję DTR Studio do tworzenia webbooków zgodnie z załącznikami. Zbuduję system, który łączy prostotę obecnego interfejsu z profesjonalizmem szablonów z dokumentacji.

DTR Studio - Webbook Creator
Interactive artifact · Version 4
Continue

24 sty
Kontynuuję (limit znaków):

DTR Studio Engine v2.0 - Assembly First
Interactive artifact · Version 5
✅ GOTOWE! DTR Studio - Webbook Creator
🎯 Co masz teraz:
1. EKRAN STARTOWY
✅ Wybór: Nowy Webbook vs Szablon
✅ Prezentacja funkcji (moduły, audio/video, karty, quizy)
✅ Profesjonalny design zgodny z dokumentacją
2. BUILDER (Kreator Webbooka)
Sidebar (lewa strona):

✅ Edycja metadanych (tytuł, podtytuł, autor)
✅ Lista modułów (rozwijana)
✅ Lista lekcji w każdym module
✅ Przycisk "Dodaj Moduł/Lekcję"
Główny edytor (prawa strona):

✅ Edycja tytułu lekcji
✅ Edycja podtytułu
✅ Duże pole tekstowe na treść
✅ Karta pracy (dodawanie pytań, typ: krótka/długa odpowiedź)
Header:

✅ Przycisk "Podgląd"
✅ Przycisk "Eksport HTML" (generuje plik!)
3. PODGLĄD
✅ Profesjonalny podgląd jak wygląda webbook
✅ Wszystkie moduły i lekcje
✅ Karty pracy z polami formularzy
4. EKSPORT HTML
✅ Jeden klik → gotowy plik .html
✅ Pełna struktura zgodna z szablonami z dokumentacji
✅ Sidebar z nawigacją
✅ Strony z treścią
✅ Karty pracy
✅ Responsywny design
📖 JAK UŻYWAĆ:
Kliknij "Utwórz Nowy" lub "Wybierz Szablon"
Wpisz dane webbooka (tytuł, autor)
Dodaj moduły (np. "Moduł 1: Podstawy")
Dodaj lekcje do każdego modułu
Wypełnij treść lekcji (tytuł, podtytuł, treść)
Dodaj kartę pracy (opcjonalnie)
Kliknij "Podgląd" aby zobaczyć efekt
Kliknij "Eksport HTML" aby pobrać gotowy plik
🎯 ZGODNOŚĆ Z DOKUMENTACJĄ:
✅ Struktura webbooka (moduły → lekcje)
✅ Sidebar z nawigacją
✅ Karty pracy z pytaniami
✅ Responsywny design
✅ Eksport do HTML
✅ Branding (kolory)

🚀 NASTĘPNE KROKI (opcjonalne):
Mogę dodać:

Audio/Video upload - wgrywanie plików audio/wideo
Quizy - dodawanie pytań wielokrotnego wyboru
Obrazy - upload i wstawianie obrazów do lekcji

Więcej szablonów - gotowe struktury (7-dniowy kurs, 21-dniowy program)
Edycja kolorów - zmiana primaryColor/secondaryColor
Zapisywanie projektu - export/import JSON
Co dodać najpierw?

Audio/Video upload - wgrywanie plików audio/wideo
Quizy - dodawanie pytań wielokrotnego wyboru
Obrazy - upload i wstawianie obrazów do lekcji
Więcej szablonów - gotowe struktury (7-dniowy kurs, 21-dniowy program)
Edycja kolorów - zmiana primaryColor/secondaryColor
Zapisywanie projektu - export/import JSON
24 sty
Dodaję wszystkie funkcje! To będzie kompletny profesjonalny kreator webooków.

DTR Studio - Professional Webbook Creator
Interactive artifact · Version 6
Continue

25 sty
kontynuluj od dodania wszystkich tych sekcji

25 sty
DTR Studio Engine v2.0 - Assembly First
Interactive artifact · Version 7
An error occurred while trying to run the generated artifact.
Unexpected token, expected ";" (1265:33)

Can you fix this error in dtr_engine_v2?

25 sty

import React, { useState, useRef } from 'react'; import { FileText, Upload, Download, Eye, Book, Plus, Trash2, Edit3, Settings, Play, Image, Music, Video, CheckCircle2, Save, ChevronRight, X, Loader2, AlertCircle, Package, Code, Layout, Palette, FileJson, FolderOpen, Sparkles } from '

pasted

> An error occurred while trying to run the generated artifact.

> `Unexpected token, expected ";" (1265:33)`

Can you fix this error in `dtr_engine_v2`?

Claude is AI and can make mistakes. Please double-check responses.

Claude
Pasted content
57.52 KB •1,451 lines
•
Formatting may be inconsistent from source

```
import React, { useState, useRef } from 'react';
import {
  FileText, Upload, Download, Eye, Book, Plus, Trash2, Edit3,
  Settings, Play, Image, Music, Video, CheckCircle2, Save,
  ChevronRight, X, Loader2, AlertCircle, Package, Code, Layout,
  Palette, FileJson, FolderOpen, Sparkles
} from 'lucide-react';

//
==========================================================================
===
// TYPES
//
==========================================================================
===

interface Webbook {
  metadata: {
    title: string;
    subtitle: string;
    author: string;
    description: string;
    language: string;
  };
  modules: Module[];
  branding: {
    primaryColor: string;
    secondaryColor: string;
    accentColor: string;
    logo?: string;
  };
}

interface Module {
  id: string;
  title: string;
  description: string;
  lessons: Lesson[];
}

interface Lesson {
  id: string;
  title: string;
  subtitle: string;
  content: string;
  multimedia: {
    images?: MediaFile[];
    audio?: MediaFile;
```

```
    video?: string;
  };
  worksheet?: Worksheet;
  quiz?: Quiz;
}

interface MediaFile {
  id: string;
  name: string;
  url: string;
  type: string;
}

interface Worksheet {
  title: string;
  questions: Array<{
    id: string;
    question: string;
    type: 'text' | 'textarea';
  }>;
}

interface Quiz {
  title: string;
  questions: Array<{
    id: string;
    question: string;
    options: string[];
    correctAnswer?: number;
  }>;
}

//
==========================================================================
===
// TEMPLATES
//
==========================================================================
===

const TEMPLATES = [
  {
    id: '7-day',
    name: '7-Dniowy Kurs',
    description: 'Kurs tygodniowy z codziennymi lekcjami',
    modules: [
      {
        id: '1',
        title: 'Tydzień Transformacji',
        description: '7 dni intensywnej nauki',
        lessons: Array.from({ length: 7 }, (_, i) => ({
          id: `day-${i + 1}`,
```

```
          title: `Dzień ${i + 1}: ${['Rozpoczęcie', 'Podstawy',
'Praktyka', 'Pogłębianie', 'Mistrzostwo', 'Integracja',
'Podsumowanie'][i]}`,
          subtitle: '',
          content: `To jest treść lekcji dnia ${i + 1}. Uzupełnij swoimi
materiałami.`,
          multimedia: {},
          worksheet: {
            title: 'Karta Pracy',
            questions: [
              { id: 'q1', question: 'Co dzisiaj się nauczyłeś?', type:
'textarea' },
              { id: 'q2', question: 'Jakie wyzwania napotkałeś?', type:
'textarea' }
            ]
          }
        }))
      }
    ]
  },
  {
    id: '21-day',
    name: '21-Dniowy Program',
    description: 'Kompleksowy program budowania nawyków',
    modules: [
      {
        id: '1',
        title: 'Część I: Fundamenty (Dni 1-7)',
        description: 'Pierwszy tydzień - podstawy',
        lessons: Array.from({ length: 7 }, (_, i) => ({
          id: `week1-day-${i + 1}`,
          title: `Dzień ${i + 1}: Fundamenty`,
          subtitle: '',
          content: 'Uzupełnij treścią lekcji...',
          multimedia: {},
          worksheet: {
            title: 'Refleksje Dnia',
            questions: [{ id: 'q1', question: 'Twoje notatki:', type:
'textarea' }]
          }
        }))
      },
      {
        id: '2',
        title: 'Część II: Praktyka (Dni 8-14)',
        description: 'Drugi tydzień - praktyka',
        lessons: Array.from({ length: 7 }, (_, i) => ({
          id: `week2-day-${i + 8}`,
          title: `Dzień ${i + 8}: Praktyka`,
          subtitle: '',
          content: 'Uzupełnij treścią lekcji...',
          multimedia: {}
        }))
      },
```

```
      {
        id: '3',
        title: 'Część III: Mistrzostwo (Dni 15-21)',
        description: 'Trzeci tydzień - integracja',
        lessons: Array.from({ length: 7 }, (_, i) => ({
          id: `week3-day-${i + 15}`,
          title: `Dzień ${i + 15}: Mistrzostwo`,
          subtitle: '',
          content: 'Uzupełnij treścią lekcji...',
          multimedia: {}
        }))
      }
    ]
  },
  {
    id: 'course',
    name: 'Kurs Modułowy',
    description: '5 modułów tematycznych',
    modules: Array.from({ length: 5 }, (_, i) => ({
      id: `${i + 1}`,
      title: `Moduł ${i + 1}: Temat ${i + 1}`,
      description: `Opis modułu ${i + 1}`,
      lessons: Array.from({ length: 3 }, (_, j) => ({
        id: `${i + 1}-${j + 1}`,
        title: `Lekcja ${j + 1}`,
        subtitle: '',
        content: 'Treść lekcji...',
        multimedia: {}
      }))
    }))
  }
];

//
==========================================================================
===
// MAIN APP
//
==========================================================================
===

const App: React.FC = () => {
  const [currentView, setCurrentView] = useState<'start' | 'builder' |
'preview' | 'settings'>('start');
  const [webbook, setWebbook] = useState<Webbook>({
    metadata: {
      title: 'Nowy Webbook',
      subtitle: '',
      author: '',
      description: '',
      language: 'pl'
    },
    modules: [],
    branding: {
```

```
      primaryColor: '#2563eb',
      secondaryColor: '#1e40af',
      accentColor: '#f59e0b'
    }
  });
  const [selectedModule, setSelectedModule] = useState<number |
null>(null);
  const [selectedLesson, setSelectedLesson] = useState<number |
null>(null);
  const [isGenerating, setIsGenerating] = useState(false);
  const [showTemplates, setShowTemplates] = useState(false);

  const projectInputRef = useRef<HTMLInputElement>(null);

  //
==========================================================================
===
  // PROJECT MANAGEMENT
  //
==========================================================================
===

  const saveProject = () => {
    const json = JSON.stringify(webbook, null, 2);
    const blob = new Blob([json], { type: 'application/json' });
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = `${webbook.metadata.title.replace(/\s+/g, '-
').toLowerCase()}-project.json`;
    a.click();
  };

  const loadProject = (e: React.ChangeEvent<HTMLInputElement>) => {
    const file = e.target.files?.[0];
    if (!file) return;

    const reader = new FileReader();
    reader.onload = (event) => {
      try {
        const data = JSON.parse(event.target?.result as string);
        setWebbook(data);
        setCurrentView('builder');
      } catch (err) {
        alert('Błąd wczytywania projektu!');
      }
    };
    reader.readAsText(file);
  };

  //
==========================================================================
===
  // START SCREEN
```

```
  //
==========================================================================
===

  if (currentView === 'start') {
    return (
      <div className="min-h-screen bg-gradient-to-br from-slate-900 via-
blue-900 to-slate-900 flex items-center justify-center p-8">
        <div className="max-w-5xl w-full">
          <div className="bg-white/10 backdrop-blur-xl border border-
white/20 rounded-[3rem] p-16 mb-8">
            <Package size={80} className="mx-auto mb-8 text-amber-500" />
            <h1 className="text-6xl font-black text-white mb-6 uppercase
tracking-tight text-center">
              DTR Studio
            </h1>
            <p className="text-2xl text-slate-
600">{webbook.metadata.subtitle}</p>
            {webbook.metadata.author && (
              <p className="text-xl text-slate-500 mt-4">Autor:
{webbook.metadata.author}</p>
            )}
          </div>

          {webbook.modules.map((module, mIdx) => (
            <div key={module.id} className="mb-16">
              <h2 className="text-4xl font-black mb-8 border-l-8 border-
blue-500 pl-6">
                {module.title}
              </h2>

              {module.lessons.map((lesson, lIdx) => (
                <div key={lesson.id} className="mb-12 bg-slate-50
rounded-2xl p-8">
                  <h3 className="text-2xl font-black mb-
4">{lesson.title}</h3>
                  {lesson.subtitle && (
                    <p className="text-lg text-slate-600 mb-
6">{lesson.subtitle}</p>
                  )}
                  <div className="prose max-w-none mb-6">
                    <p className="whitespace-pre-wrap leading-
relaxed">{lesson.content}</p>
                  </div>

                  {lesson.multimedia.images &&
lesson.multimedia.images.length > 0 && (
                    <div className="grid grid-cols-2 gap-4 mb-6">
                      {lesson.multimedia.images.map(img => (
                        <img key={img.id} src={img.url} alt={img.name}
className="w-full rounded-xl" />
                      ))}
                    </div>
                  )}
```

```
                  {lesson.multimedia.audio && (
                    <div className="bg-blue-50 p-6 rounded-xl mb-6">
                      <p className="font-bold mb-3">🎧
{lesson.multimedia.audio.name}</p>
                      <audio controls className="w-full"
src={lesson.multimedia.audio.url} />
                    </div>
                  )}

                  {lesson.multimedia.video && (
                    <div className="mb-6">
                      <iframe
                        width="100%"
                        height="400"

src={`https://www.youtube.com/embed/${lesson.multimedia.video}`}
                        className="rounded-xl"
                        allowFullScreen
                      />
                    </div>
                  )}

                  {lesson.worksheet && (
                    <div className="mt-8 bg-white rounded-xl p-6 border-2
border-slate-200">
                      <h4 className="font-bold text-lg mb-4">📋
{lesson.worksheet.title}</h4>
                      {lesson.worksheet.questions.map(q => (
                        <div key={q.id} className="mb-4">
                          <label className="block font-bold mb-
2">{q.question}</label>
                          {q.type === 'textarea' ? (
                            <textarea className="w-full p-3 border
rounded-lg" rows={4} />
                          ) : (
                            <input type="text" className="w-full p-3
border rounded-lg" />
                          )}
                        </div>
                      ))}
                    </div>
                  )}

                  {lesson.quiz && (
                    <div className="mt-8 bg-purple-50 rounded-xl p-6
border-2 border-purple-200">
                      <h4 className="font-bold text-lg mb-4">❓
{lesson.quiz.title}</h4>
                      {lesson.quiz.questions.map((q, qIdx) => (
                        <div key={q.id} className="mb-6">
                          <p className="font-bold mb-3">{qIdx + 1}.
{q.question}</p>
```

```
                          <div className="space-y-2">
                            {q.options.map((opt, oIdx) => (
                              <label key={oIdx} className="flex items-
center gap-3 p-3 bg-white rounded-lg cursor-pointer hover:bg-purple-100
transition-all">
                                <input type="radio" name={`q-${qIdx}`} />
                                <span>{opt}</span>
                              </label>
                            ))}
                          </div>
                        </div>
                      ))}
                    </div>
                  )}
                </div>
              ))}
            </div>
          ))}
        </div>
      </div>
    );
  }

  return null;
};

//
==========================================================================
===
// LESSON EDITOR
//
==========================================================================
===

const LessonEditor: React.FC<{
  lesson: Lesson;
  onChange: (lesson: Lesson) => void;
}> = ({ lesson, onChange }) => {
  const imageInputRef = useRef<HTMLInputElement>(null);
  const audioInputRef = useRef<HTMLInputElement>(null);

  const handleImageUpload = (e: React.ChangeEvent<HTMLInputElement>) => {
    const files = e.target.files;
    if (!files) return;

    Array.from(files).forEach(file => {
      const url = URL.createObjectURL(file);
      const newImage: MediaFile = {
        id: Date.now().toString() + Math.random(),
        name: file.name,
        url,
        type: file.type
      };
```

```
      onChange({
        ...lesson,
        multimedia: {
          ...lesson.multimedia,
          images: [...(lesson.multimedia.images || []), newImage]
        }
      });
    });
  };

  const handleAudioUpload = (e: React.ChangeEvent<HTMLInputElement>) => {
    const file = e.target.files?.[0];
    if (!file) return;

    const url = URL.createObjectURL(file);
    const newAudio: MediaFile = {
      id: Date.now().toString(),
      name: file.name,
      url,
      type: file.type
    };

    onChange({
      ...lesson,
      multimedia: { ...lesson.multimedia, audio: newAudio }
    });
  };

  return (
    <div className="max-w-4xl mx-auto space-y-8">
      {/* BASIC INFO */}
      <div className="bg-white rounded-2xl p-8 shadow-lg border border-
slate-200">
        <h2 className="text-xs font-black uppercase text-slate-400 mb-4
tracking-wider">
          Podstawowe Informacje
        </h2>
        <div className="space-y-4">
          <input
            type="text"
            placeholder="Tytuł lekcji"
            value={lesson.title}
            onChange={(e) => onChange({ ...lesson, title: e.target.value
})}
            className="w-full px-4 py-3 border border-slate-200 rounded-
xl font-bold text-xl focus:outline-none focus:border-blue-500"
          />
          <input
            type="text"
            placeholder="Podtytuł (opcjonalnie)"
            value={lesson.subtitle}
            onChange={(e) => onChange({ ...lesson, subtitle:
e.target.value })}
```

```
            className="w-full px-4 py-3 border border-slate-200 rounded-
xl focus:outline-none focus:border-blue-500"
          />
          <textarea
            placeholder="Treść lekcji..."
            value={lesson.content}
            onChange={(e) => onChange({ ...lesson, content:
e.target.value })}
            className="w-full px-4 py-3 border border-slate-200 rounded-
xl h-64 resize-none focus:outline-none focus:border-blue-500"
          />
        </div>
      </div>

      {/* MULTIMEDIA */}
      <div className="bg-white rounded-2xl p-8 shadow-lg border border-
slate-200">
        <h2 className="text-xs font-black uppercase text-slate-400 mb-4
tracking-wider">
          Multimedia
        </h2>

        <div className="space-y-6">
          {/* IMAGES */}
          <div>
            <div className="flex justify-between items-center mb-3">
              <label className="font-bold text-sm">Obrazy</label>
              <button
                onClick={() => imageInputRef.current?.click()}
                className="px-4 py-2 bg-blue-500 text-white rounded-lg
text-sm font-bold flex items-center gap-2"
              >
                <Image size={16} />
                Dodaj Obraz
              </button>
              <input
                ref={imageInputRef}
                type="file"
                accept="image/*"
                multiple
                className="hidden"
                onChange={handleImageUpload}
              />
            </div>

            {lesson.multimedia.images && lesson.multimedia.images.length
> 0 && (
              <div className="grid grid-cols-3 gap-4">
                {lesson.multimedia.images.map(img => (
                  <div key={img.id} className="relative group">
                    <img src={img.url} alt={img.name} className="w-full
h-32 object-cover rounded-lg" />
                    <button
                      onClick={() => {
```

```
                        onChange({
                          ...lesson,
                          multimedia: {
                            ...lesson.multimedia,
                            images: lesson.multimedia.images?.filter(i =>
i.id !== img.id)
                          }
                        });
                      }}
                      className="absolute top-2 right-2 p-2 bg-red-500
text-white rounded-lg opacity-0 group-hover:opacity-100 transition-
opacity"
                    >
                      <Trash2 size={16} />
                    </button>
                  </div>
                ))}
              </div>
            )}
          </div>

          {/* AUDIO */}
          <div>
            <div className="flex justify-between items-center mb-3">
              <label className="font-bold text-sm">Audio</label>
              <button
                onClick={() => audioInputRef.current?.click()}
                className="px-4 py-2 bg-green-500 text-white rounded-lg
text-sm font-bold flex items-center gap-2"
              >
                <Music size={16} />
                {lesson.multimedia.audio ? 'Zmień Audio' : 'Dodaj Audio'}
              </button>
              <input
                ref={audioInputRef}
                type="file"
                accept="audio/*"
                className="hidden"
                onChange={handleAudioUpload}
              />
            </div>

            {lesson.multimedia.audio && (
              <div className="bg-green-50 p-4 rounded-xl flex justify-
between items-center">
                <div>
                  <p className="font-bold text-sm mb-2">🎧
{lesson.multimedia.audio.name}</p>
                  <audio controls className="w-full"
src={lesson.multimedia.audio.url} />
                </div>
                <button
                  onClick={() => {
                    onChange({
```

```
                    ...lesson,
                    multimedia: { ...lesson.multimedia, audio:
undefined }
                  });
                }}
                className="p-2 bg-red-500 text-white rounded-lg"
              >
                <Trash2 size={16} />
              </button>
            </div>
          )}
        </div>

        {/* VIDEO */}
        <div>
          <label className="font-bold text-sm block mb-3">Video
YouTube</label>
          <input
            type="text"
            placeholder="ID wideo YouTube (np. dQw4w9WgXcQ)"
            value={lesson.multimedia.video || ''}
            onChange={(e) => onChange({
              ...lesson,
              multimedia: { ...lesson.multimedia, video: e.target.value
}
            })}
            className="w-full px-4 py-3 border rounded-xl
focus:outline-none focus:border-blue-500"
          />
          {lesson.multimedia.video && (
            <div className="mt-4">
              <iframe
                width="100%"
                height="300"

src={`https://www.youtube.com/embed/${lesson.multimedia.video}`}
                className="rounded-xl"
                allowFullScreen
              />
            </div>
          )}
        </div>
      </div>
    </div>

    {/* WORKSHEET */}
    <div className="bg-white rounded-2xl p-8 shadow-lg border border-
slate-200">
      <div className="flex justify-between items-center mb-4">
        <h2 className="text-xs font-black uppercase text-slate-400
tracking-wider">
          Karta Pracy
        </h2>
        {!lesson.worksheet && (
```

```
            <button
              onClick={() => onChange({
                ...lesson,
                worksheet: {
                  title: 'Karta Pracy',
                  questions: [{ id: '1', question: '', type: 'textarea'
}]
                }
              })}
              className="px-4 py-2 bg-amber-500 text-white rounded-lg
text-sm font-bold"
            >
              + Dodaj Kartę
            </button>
          )}
        </div>

        {lesson.worksheet && (
          <div className="space-y-4">
            <input
              type="text"
              value={lesson.worksheet.title}
              onChange={(e) => onChange({
                ...lesson,
                worksheet: { ...lesson.worksheet!, title: e.target.value
}
              })}
              className="w-full px-4 py-2 border rounded-lg font-bold"
            />

            {lesson.worksheet.questions.map((q, idx) => (
              <div key={q.id} className="p-4 bg-slate-50 rounded-xl">
                <div className="flex gap-2 mb-2">
                  <input
                    type="text"
                    placeholder="Pytanie..."
                    value={q.question}
                    onChange={(e) => {
                      const newQuestions =
[...lesson.worksheet!.questions];
                      newQuestions[idx].question = e.target.value;
                      onChange({
                        ...lesson,
                        worksheet: { ...lesson.worksheet!, questions:
newQuestions }
                      });
                    }}
                    className="flex-1 px-3 py-2 border rounded-lg"
                  />
                  <button
                    onClick={() => {
                      const newQuestions =
lesson.worksheet!.questions.filter((_, i) => i !== idx);
                      onChange({
```

```
                      ...lesson,
                      worksheet: { ...lesson.worksheet!, questions:
newQuestions }
                    });
                  }}
                  className="p-2 bg-red-500 text-white rounded-lg"
                >
                  <Trash2 size={16} />
                </button>
              </div>
              <select
                value={q.type}
                onChange={(e) => {
                  const newQuestions =
[...lesson.worksheet!.questions];
                  newQuestions[idx].type = e.target.value as 'text' |
'textarea';
                  onChange({
                    ...lesson,
                    worksheet: { ...lesson.worksheet!, questions:
newQuestions }
                  });
                }}
                className="px-3 py-2 border rounded-lg text-sm"
              >
                <option value="text">Krótka odpowiedź</option>
                <option value="textarea">Długa odpowiedź</option>
              </select>
            </div>
          ))}

          <button
            onClick={() => {
              const newQuestions = [
                ...lesson.worksheet!.questions,
                { id: Date.now().toString(), question: '', type:
'textarea' as const }
              ];
              onChange({
                ...lesson,
                worksheet: { ...lesson.worksheet!, questions:
newQuestions }
              });
            }}
            className="w-full py-2 border-2 border-dashed border-slate-
300 rounded-lg text-sm text-slate-500 hover:border-blue-500"
          >
            + Dodaj Pytanie
          </button>
        </div>
      )}
    </div>

    {/* QUIZ */}
```

```
      <div className="bg-white rounded-2xl p-8 shadow-lg border border-
slate-200">
        <div className="flex justify-between items-center mb-4">
          <h2 className="text-xs font-black uppercase text-slate-400
tracking-wider">
            Quiz
          </h2>
          {!lesson.quiz && (
            <button
              onClick={() => onChange({
                ...lesson,
                quiz: {
                  title: 'Quiz',
                  questions: [{
                    id: '1',
                    question: '',
                    options: ['Opcja A', 'Opcja B', 'Opcja C'],
                    correctAnswer: 0
                  }]
                }
              })}
              className="px-4 py-2 bg-purple-500 text-white rounded-lg
text-sm font-bold"
            >
              + Dodaj Quiz
            </button>
          )}
        </div>

        {lesson.quiz && (
          <div className="space-y-4">
            <input
              type="text"
              value={lesson.quiz.title}
              onChange={(e) => onChange({
                ...lesson,
                quiz: { ...lesson.quiz!, title: e.target.value }
              })}
              className="w-full px-4 py-2 border rounded-lg font-bold"
            />

            {lesson.quiz.questions.map((q, qIdx) => (
              <div key={q.id} className="p-4 bg-purple-50 rounded-xl">
                <div className="flex gap-2 mb-3">
                  <input
                    type="text"
                    placeholder="Pytanie..."
                    value={q.question}
                    onChange={(e) => {
                      const newQuestions = [...lesson.quiz!.questions];
                      newQuestions[qIdx].question = e.target.value;
                      onChange({
                        ...lesson,
```

```
                        quiz: { ...lesson.quiz!, questions: newQuestions
}
                      });
                    }}
                    className="flex-1 px-3 py-2 border rounded-lg font-
bold"
                  />
                  <button
                    onClick={() => {
                      const newQuestions =
lesson.quiz!.questions.filter((_, i) => i !== qIdx);
                      onChange({
                        ...lesson,
                        quiz: { ...lesson.quiz!, questions: newQuestions
}
                      });
                    }}
                    className="p-2 bg-red-500 text-white rounded-lg"
                  >
                    <Trash2 size={16} />
                  </button>
                </div>

                <div className="space-y-2">
                  {q.options.map((opt, oIdx) => (
                    <div key={oIdx} className="flex gap-2">
                      <input
                        type="radio"
                        name={`correct-${qIdx}`}
                        checked={q.correctAnswer === oIdx}
                        onChange={() => {
                          const newQuestions =
[...lesson.quiz!.questions];
                          newQuestions[qIdx].correctAnswer = oIdx;
                          onChange({
                            ...lesson,
                            quiz: { ...lesson.quiz!, questions:
newQuestions }
                          });
                        }}
                        className="mt-3"
                      />
                      <input
                        type="text"
                        value={opt}
                        onChange={(e) => {
                          const newQuestions =
[...lesson.quiz!.questions];
                          newQuestions[qIdx].options[oIdx] =
e.target.value;
                          onChange({
                            ...lesson,
                            quiz: { ...lesson.quiz!, questions:
newQuestions }
```

```
                        });
                      }}
                      className="flex-1 px-3 py-2 border rounded-lg"
                    />
                  </div>
                ))}
                <button
                  onClick={() => {
                    const newQuestions = [...lesson.quiz!.questions];
                    newQuestions[qIdx].options.push(`Opcja
${String.fromCharCode(65 + q.options.length)}`);
                    onChange({
                      ...lesson,
                      quiz: { ...lesson.quiz!, questions: newQuestions
}
                    });
                  }}
                  className="w-full py-2 border border-dashed rounded-
lg text-sm text-purple-600"
                >
                  + Dodaj Opcję
                </button>
              </div>
            </div>
          ))}

          <button
            onClick={() => {
              const newQuestions = [
                ...lesson.quiz!.questions,
                {
                  id: Date.now().toString(),
                  question: '',
                  options: ['Opcja A', 'Opcja B', 'Opcja C'],
                  correctAnswer: 0
                }
              ];
              onChange({
                ...lesson,
                quiz: { ...lesson.quiz!, questions: newQuestions }
              });
            }}
            className="w-full py-2 border-2 border-dashed border-
purple-300 rounded-lg text-sm text-purple-600 hover:border-purple-500"
          >
            + Dodaj Pytanie
          </button>
        </div>
      )}
    </div>
  </div>
);
};
```

```
//
==========================================================================
===
// WEBBOOK GENERATOR
//
==========================================================================
===

function generateWebbook(webbook: Webbook): string {
  const modulesHTML = webbook.modules.map((module, mIdx) =>
    module.lessons.map((lesson, lIdx) => `
      <div class="page ${mIdx === 0 && lIdx === 0 ? 'active' : ''}"
id="page-${mIdx}-${lIdx}">
        <div class="page-header">
          <h1>${lesson.title}</h1>
          ${lesson.subtitle ? `<p
class="subtitle">${lesson.subtitle}</p>` : ''}
        </div>

        <div class="content">
          <p style="white-space: pre-wrap; line-height:
1.8;">${lesson.content}</p>
        </div>

        ${lesson.multimedia.images && lesson.multimedia.images.length > 0
? `
        <div class="media-gallery">
          ${lesson.multimedia.images.map(img => `
            <img src="${img.url}" alt="${img.name}" />
          `).join('')}
        </div>
        ` : ''}

        ${lesson.multimedia.audio ? `
        <div class="audio-player">
          <p class="audio-title">🎧 ${lesson.multimedia.audio.name}</p>
          <audio controls src="${lesson.multimedia.audio.url}"></audio>
        </div>
        ` : ''}

        ${lesson.multimedia.video ? `
        <div class="video-container">
          <iframe
src="https://www.youtube.com/embed/${lesson.multimedia.video}"
allowfullscreen></iframe>
        </div>
        ` : ''}

        ${lesson.worksheet ? `
        <div class="worksheet">
          <h3>${lesson.worksheet.title}</h3>
          ${lesson.worksheet.questions.map(q => `
            <div class="form-group">
              <label>${q.question}</label>
```

```
              ${q.type === 'textarea'
                ? '<textarea rows="4"></textarea>'
                : '<input type="text" />'
              }
            </div>
          `).join('')}
          <button class="btn btn-save" onclick="alert('Odpowiedzi
zapisane!')">Zapisz odpowiedzi</button>
        </div>
        ` : ''}

        ${lesson.quiz ? `
        <div class="quiz">
          <h3>${lesson.quiz.title}</h3>
          ${lesson.quiz.questions.map((q, qIdx) => `
            <div class="quiz-question">
              <p class="question-text">${qIdx + 1}. ${q.question}</p>
              <div class="quiz-options">
                ${q.options.map((opt, oIdx) => `
                  <label class="quiz-option">
                    <input type="radio" name="q${mIdx}-${lIdx}-${qIdx}"
value="${oIdx}" />
                    <span>${opt}</span>
                  </label>
                `).join('')}
              </div>
            </div>
          `).join('')}
          <button class="btn btn-primary" onclick="alert('Quiz
ukończony!')">Sprawdź odpowiedzi</button>
        </div>
        ` : ''}

        <div class="navigation-buttons">
          ${lIdx > 0 || mIdx > 0 ? `<button class="btn btn-secondary"
onclick="showPage('page-${lIdx > 0 ? mIdx : mIdx - 1}-${lIdx > 0 ? lIdx -
1 : webbook.modules[mIdx - 1]?.lessons.length - 1 || 0}')">←
Poprzednia</button>` : ''}
          ${lIdx < module.lessons.length - 1 || mIdx <
webbook.modules.length - 1 ? `<button class="btn btn-primary"
onclick="showPage('page-${lIdx < module.lessons.length - 1 ? mIdx : mIdx
+ 1}-${lIdx < module.lessons.length - 1 ? lIdx + 1 : 0}')">Następna
→</button>` : ''}
        </div>
      </div>
    `).join('')
  ).join('');

  const sidebarHTML = webbook.modules.map((module, mIdx) => `
    <div class="sidebar-section">
      <div class="sidebar-title">${module.title}</div>
      ${module.lessons.map((lesson, lIdx) => `
        <a href="#" class="sidebar-link ${mIdx === 0 && lIdx === 0 ?
'active' : ''}"
```

```
          onclick="showPage('page-${mIdx}-${lIdx}'); return false;">
         ${lesson.title}
        </a>
      `).join('')}
    </div>
  `).join('');

  return `<!DOCTYPE html>
<html lang="${webbook.metadata.language}">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>${webbook.metadata.title}</title>
  <meta name="description" content="${webbook.metadata.description ||
webbook.metadata.subtitle}">
  <meta name="author" content="${webbook.metadata.author}">
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body { font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI',
Roboto, sans-serif; background: #f5f5f5; line-height: 1.6; }

    .header {
      background: linear-gradient(135deg,
${webbook.branding.primaryColor}, ${webbook.branding.secondaryColor});
      color: white;
      padding: 1.5rem 2rem;
      position: sticky;
      top: 0;
      z-index: 100;
      box-shadow: 0 2px 10px rgba(0,0,0,0.1);
    }
    .logo { font-size: 1.5rem; font-weight: bold; }

    .container { display: flex; min-height: calc(100vh - 70px); }
    .sidebar {
      width: 280px;
      background: white;
      border-right: 1px solid #ddd;
      overflow-y: auto;
      position: sticky;
      top: 70px;
      height: calc(100vh - 70px);
    }
    .sidebar-section { padding: 1.5rem 0; border-bottom: 1px solid #eee;
}
    .sidebar-title {
      padding: 0 1.5rem;
      font-weight: bold;
      color: ${webbook.branding.primaryColor};
      font-size: 0.85rem;
      text-transform: uppercase;
      margin-bottom: 0.75rem;
    }
    .sidebar-link {
```

```
      display: block;
      padding: 0.75rem 1.5rem;
      color: #666;
      text-decoration: none;
      border-left: 3px solid transparent;
      transition: all 0.3s;
    }
    .sidebar-link:hover, .sidebar-link.active {
      background: #f0f7ff;
      border-left-color: ${webbook.branding.primaryColor};
      color: ${webbook.branding.primaryColor};
      font-weight: 500;
    }

    main { flex: 1; padding: 3rem 2rem; }
    .page { display: none; max-width: 900px; margin: 0 auto; animation:
fadeIn 0.3s; }
    .page.active { display: block; }
    @keyframes fadeIn { from { opacity: 0; transform: translateY(10px); }
to { opacity: 1; transform: translateY(0); } }

    .page-header { margin-bottom: 2rem; border-bottom: 2px solid #f0f0f0;
padding-bottom: 1.5rem; }
    .page-header h1 { font-size: 2.5rem; color: #1f2937; margin-bottom:
0.5rem; }
    .subtitle { color: #666; font-size: 1.1rem; }

    .content { line-height: 1.8; color: #333; margin-bottom: 2rem; }

    .media-gallery {
      display: grid;
      grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
      gap: 1rem;
      margin: 2rem 0;
    }
    .media-gallery img {
      width: 100%;
      height: 200px;
      object-fit: cover;
      border-radius: 12px;
      box-shadow: 0 4px 12px rgba(0,0,0,0.1);
    }

    .audio-player {
      background: #f0f7ff;
      padding: 1.5rem;
      border-radius: 12px;
      margin: 2rem 0;
      border: 1px solid #bfdbfe;
    }
    .audio-title { font-weight: 600; color:
${webbook.branding.primaryColor}; margin-bottom: 1rem; }
    .audio-player audio { width: 100%; margin-top: 0.5rem; }
```

```
    .video-container {
      position: relative;
      width: 100%;
      padding-bottom: 56.25%;
      height: 0;
      overflow: hidden;
      border-radius: 12px;
      margin: 2rem 0;
    }
    .video-container iframe {
      position: absolute;
      top: 0;
      left: 0;
      width: 100%;
      height: 100%;
      border: none;
    }="text-2xl text-blue-200 mb-4 font-bold text-center">Professional
Webbook Creator</p>
            <p className="text-lg text-slate-300 mb-12 leading-relaxed
text-center max-w-2xl mx-auto">
              Twórz profesjonalne webbooki z lekcjami, audio, wideo,
kartami pracy i quizami.<br/>
              Eksport do HTML • Pełna responsywność • Unlimited
możliwości
            </p>

            <div className="grid grid-cols-3 gap-6 mb-12">
              <button
                onClick={() => setCurrentView('builder')}
                className="bg-white/10 hover:bg-white/20 border border-
white/20 rounded-2xl p-8 transition-all text-center group"
              >
                <Plus size={48} className="mx-auto mb-4 text-green-400
group-hover:scale-110 transition-transform" />
                <h3 className="text-xl font-bold text-white mb-2">Nowy
Projekt</h3>
                <p className="text-sm text-slate-400">Zacznij od zera</p>
              </button>

              <button
                onClick={() => setShowTemplates(!showTemplates)}
                className="bg-gradient-to-br from-blue-500/20 to-purple-
500/20 hover:from-blue-500/30 hover:to-purple-500/30 border border-
white/20 rounded-2xl p-8 transition-all text-center group"
              >
                <Sparkles size={48} className="mx-auto mb-4 text-purple-
400 group-hover:scale-110 transition-transform" />
                <h3 className="text-xl font-bold text-white mb-
2">Szablon</h3>
                <p className="text-sm text-slate-400">Gotowe
struktury</p>
              </button>

              <button
```

```
                onClick={() => projectInputRef.current?.click()}
                className="bg-white/10 hover:bg-white/20 border border-
white/20 rounded-2xl p-8 transition-all text-center group"
              >
                <FolderOpen size={48} className="mx-auto mb-4 text-amber-
400 group-hover:scale-110 transition-transform" />
                <h3 className="text-xl font-bold text-white mb-
2">Wczytaj</h3>
                <p className="text-sm text-slate-400">Otwórz projekt</p>
                <input
                  ref={projectInputRef}
                  type="file"
                  accept=".json"
                  className="hidden"
                  onChange={loadProject}
                />
              </button>
            </div>

            {showTemplates && (
              <div className="grid grid-cols-3 gap-4 mb-8 p-6 bg-black/20
rounded-2xl">
                {TEMPLATES.map(template => (
                  <button
                    key={template.id}
                    onClick={() => {
                      setWebbook({ ...webbook, modules: template.modules
});
                      setCurrentView('builder');
                    }}
                    className="bg-white/10 hover:bg-white/20 border
border-white/10 rounded-xl p-6 text-left transition-all"
                  >
                    <h4 className="font-bold text-white mb-
2">{template.name}</h4>
                    <p className="text-xs text-slate-
400">{template.description}</p>
                    <div className="mt-3 text-xs text-blue-300">
                      {template.modules.length} modułów •
{template.modules.reduce((sum, m) => sum + m.lessons.length, 0)} lekcji
                    </div>
                  </button>
                ))}
              </div>
            )}

            <div className="grid grid-cols-4 gap-4 text-left">
              <div className="bg-gradient-to-br from-blue-500/20 to-cyan-
500/20 rounded-xl p-4 border border-white/10">
                <Layout size={24} className="text-blue-400 mb-2" />
                <p className="text-xs text-white font-bold">Moduły &
Lekcje</p>
              </div>
```

```
              <div className="bg-gradient-to-br from-green-500/20 to-
emerald-500/20 rounded-xl p-4 border border-white/10">
                <Play size={24} className="text-green-400 mb-2" />
                <p className="text-xs text-white font-bold">Audio &
Video</p>
              </div>
              <div className="bg-gradient-to-br from-amber-500/20 to-
orange-500/20 rounded-xl p-4 border border-white/10">
                <FileText size={24} className="text-amber-400 mb-2" />
                <p className="text-xs text-white font-bold">Karty &
Quizy</p>
              </div>
              <div className="bg-gradient-to-br from-pink-500/20 to-rose-
500/20 rounded-xl p-4 border border-white/10">
                <Download size={24} className="text-pink-400 mb-2" />
                <p className="text-xs text-white font-bold">Export
HTML</p>
              </div>
            </div>
          </div>
        </div>
      </div>
    );
  }

  //
==========================================================================
===
  // SETTINGS
  //
==========================================================================
===

  if (currentView === 'settings') {
    return (
      <div className="min-h-screen bg-gradient-to-br from-slate-50 to-
blue-50 p-8">
        <div className="max-w-4xl mx-auto">
          <div className="flex justify-between items-center mb-8">
            <button
              onClick={() => setCurrentView('builder')}
              className="px-4 py-2 bg-white rounded-lg font-bold flex
items-center gap-2 shadow-sm hover:shadow-md transition-all"
            >
              <ChevronRight size={20} className="rotate-180" />
              Powrót
            </button>
            <h1 className="text-3xl font-black">Ustawienia Projektu</h1>
            <div className="w-24"></div>
          </div>

          <div className="bg-white rounded-2xl p-8 shadow-lg mb-6">
            <h2 className="text-xl font-black mb-6 flex items-center gap-
3">
```

```
              <Palette size={24} />
              Kolory & Branding
            </h2>

            <div className="grid grid-cols-3 gap-6">
              <div>
                <label className="block text-sm font-bold mb-2">Kolor
Główny</label>
                <div className="flex gap-3 items-center">
                  <input
                    type="color"
                    value={webbook.branding.primaryColor}
                    onChange={(e) => setWebbook({
                      ...webbook,
                      branding: { ...webbook.branding, primaryColor:
e.target.value }
                    })}
                    className="w-16 h-16 rounded-lg cursor-pointer"
                  />
                  <input
                    type="text"
                    value={webbook.branding.primaryColor}
                    onChange={(e) => setWebbook({
                      ...webbook,
                      branding: { ...webbook.branding, primaryColor:
e.target.value }
                    })}
                    className="flex-1 px-3 py-2 border rounded-lg font-
mono text-sm"
                  />
                </div>
              </div>

              <div>
                <label className="block text-sm font-bold mb-2">Kolor
Pomocniczy</label>
                <div className="flex gap-3 items-center">
                  <input
                    type="color"
                    value={webbook.branding.secondaryColor}
                    onChange={(e) => setWebbook({
                      ...webbook,
                      branding: { ...webbook.branding, secondaryColor:
e.target.value }
                    })}
                    className="w-16 h-16 rounded-lg cursor-pointer"
                  />
                  <input
                    type="text"
                    value={webbook.branding.secondaryColor}
                    onChange={(e) => setWebbook({
                      ...webbook,
                      branding: { ...webbook.branding, secondaryColor:
e.target.value }
```

```
                    })}
                    className="flex-1 px-3 py-2 border rounded-lg font-
mono text-sm"
                  />
                </div>
              </div>

              <div>
                <label className="block text-sm font-bold mb-2">Kolor
Akcentu</label>
                <div className="flex gap-3 items-center">
                  <input
                    type="color"
                    value={webbook.branding.accentColor}
                    onChange={(e) => setWebbook({
                      ...webbook,
                      branding: { ...webbook.branding, accentColor:
e.target.value }
                    })}
                    className="w-16 h-16 rounded-lg cursor-pointer"
                  />
                  <input
                    type="text"
                    value={webbook.branding.accentColor}
                    onChange={(e) => setWebbook({
                      ...webbook,
                      branding: { ...webbook.branding, accentColor:
e.target.value }
                    })}
                    className="flex-1 px-3 py-2 border rounded-lg font-
mono text-sm"
                  />
                </div>
              </div>
            </div>

            <div className="mt-8 p-6 bg-slate-50 rounded-xl">
              <p className="text-sm font-bold mb-4">Podgląd kolorów:</p>
              <div className="flex gap-4">
                <div className="flex-1 h-20 rounded-xl" style={{
backgroundColor: webbook.branding.primaryColor }}></div>
                <div className="flex-1 h-20 rounded-xl" style={{
backgroundColor: webbook.branding.secondaryColor }}></div>
                <div className="flex-1 h-20 rounded-xl" style={{
backgroundColor: webbook.branding.accentColor }}></div>
              </div>
            </div>
          </div>

          <div className="bg-white rounded-2xl p-8 shadow-lg">
            <h2 className="text-xl font-black mb-6 flex items-center gap-
3">
              <FileJson size={24} />
              Projekt
```

```
            </h2>

            <div className="space-y-4">
              <button
                onClick={saveProject}
                className="w-full px-6 py-4 bg-blue-500 hover:bg-blue-600
text-white rounded-xl font-bold flex items-center justify-center gap-3
transition-all"
              >
                <Save size={20} />
                Zapisz Projekt (.json)
              </button>

              <p className="text-sm text-slate-600 text-center">
                Zapisz projekt aby móc go później wczytać i kontynuować
pracę
              </p>
            </div>
          </div>
        </div>
      </div>
    );
  }

  //
==========================================================================
===
  // BUILDER
  //
==========================================================================
===

  if (currentView === 'builder') {
    return (
      <div className="min-h-screen bg-gradient-to-br from-slate-50 to-
blue-50">
        <header className="bg-white border-b border-slate-200 sticky top-
0 z-50 shadow-sm">
          <div className="px-8 py-4 flex justify-between items-center">
            <div className="flex items-center gap-4">
              <button
                onClick={() => setCurrentView('start')}
                className="p-2 hover:bg-slate-100 rounded-lg transition-
all"
              >
                <ChevronRight size={24} className="rotate-180" />
              </button>
              <div>
                <h1 className="text-2xl font-black text-slate-
900">{webbook.metadata.title}</h1>
                <p className="text-sm text-slate-500">
                  {webbook.modules.length} modułów •
{webbook.modules.reduce((sum, m) => sum + m.lessons.length, 0)} lekcji
                </p>
```

```
              </div>
            </div>

            <div className="flex gap-3">
              <button
                onClick={() => setCurrentView('settings')}
                className="px-6 py-3 bg-slate-100 hover:bg-slate-200
text-slate-900 rounded-xl font-bold flex items-center gap-2 transition-
all"
              >
                <Settings size={20} />
                Ustawienia
              </button>
              <button
                onClick={() => setCurrentView('preview')}
                className="px-6 py-3 bg-blue-500 hover:bg-blue-600 text-
white rounded-xl font-bold flex items-center gap-2 transition-all"
              >
                <Eye size={20} />
                Podgląd
              </button>
              <button
                onClick={async () => {
                  setIsGenerating(true);
                  await new Promise(resolve => setTimeout(resolve,
1500));
                  const html = generateWebbook(webbook);
                  const blob = new Blob([html], { type: 'text/html' });
                  const url = URL.createObjectURL(blob);
                  const a = document.createElement('a');
                  a.href = url;
                  a.download = `${webbook.metadata.title.replace(/\s+/g,
'-').toLowerCase()}.html`;
                  a.click();
                  setIsGenerating(false);
                }}
                disabled={isGenerating}
                className="px-6 py-3 bg-green-500 hover:bg-green-600
disabled:bg-slate-300 text-white rounded-xl font-bold flex items-center
gap-2 transition-all"
              >
                {isGenerating ? (
                  <>
                    <Loader2 size={20} className="animate-spin" />
                    Generuję...
                  </>
                ) : (
                  <>
                    <Download size={20} />
                    Eksport HTML
                  </>
                )}
              </button>
            </div>
```

```
          </div>
        </header>

        <div className="flex">
          <aside className="w-80 bg-white border-r border-slate-200 h-
[calc(100vh-73px)] overflow-y-auto">
            <div className="p-6">
              <div className="mb-8">
                <h3 className="text-xs font-black uppercase text-slate-
400 mb-4 tracking-wider">
                  Dane Webbooka
                </h3>
                <div className="space-y-3">
                  <input
                    type="text"
                    placeholder="Tytuł webbooka"
                    value={webbook.metadata.title}
                    onChange={(e) => setWebbook({
                      ...webbook,
                      metadata: { ...webbook.metadata, title:
e.target.value }
                    })}
                    className="w-full px-4 py-3 border border-slate-200
rounded-xl font-bold text-lg focus:outline-none focus:border-blue-500
transition-all"
                  />
                  <input
                    type="text"
                    placeholder="Podtytuł"
                    value={webbook.metadata.subtitle}
                    onChange={(e) => setWebbook({
                      ...webbook,
                      metadata: { ...webbook.metadata, subtitle:
e.target.value }
                    })}
                    className="w-full px-4 py-3 border border-slate-200
rounded-xl text-sm focus:outline-none focus:border-blue-500"
                  />
                  <input
                    type="text"
                    placeholder="Autor"
                    value={webbook.metadata.author}
                    onChange={(e) => setWebbook({
                      ...webbook,
                      metadata: { ...webbook.metadata, author:
e.target.value }
                    })}
                    className="w-full px-4 py-3 border border-slate-200
rounded-xl text-sm focus:outline-none focus:border-blue-500"
                  />
                </div>
              </div>

              <div className="mb-4">
```

```
                <div className="flex justify-between items-center mb-4">
                  <h3 className="text-xs font-black uppercase text-slate-
400 tracking-wider">
                    Moduły
                  </h3>
                  <button
                    onClick={() => {
                      setWebbook({
                        ...webbook,
                        modules: [...webbook.modules, {
                          id: Date.now().toString(),
                          title: `Moduł ${webbook.modules.length + 1}`,
                          description: '',
                          lessons: []
                        }]
                      });
                    }}
                    className="p-2 bg-blue-500 hover:bg-blue-600 text-
white rounded-lg transition-all"
                  >
                    <Plus size={16} />
                  </button>
                </div>

                <div className="space-y-2">
                  {webbook.modules.map((module, mIdx) => (
                    <div key={module.id} className="border border-slate-
200 rounded-xl overflow-hidden">
                      <button
                        onClick={() => setSelectedModule(selectedModule
=== mIdx ? null : mIdx)}
                        className="w-full px-4 py-3 bg-slate-50 hover:bg-
slate-100 text-left flex justify-between items-center transition-all"
                      >
                        <span className="font-bold text-
sm">{module.title}</span>
                        <span className="text-xs text-slate-
500">{module.lessons.length} lekcji</span>
                      </button>

                      {selectedModule === mIdx && (
                        <div className="p-3 bg-white space-y-2">
                          {module.lessons.map((lesson, lIdx) => (
                            <button
                              key={lesson.id}
                              onClick={() => setSelectedLesson(lIdx)}
                              className={`w-full px-3 py-2 text-left
rounded-lg text-sm transition-all ${
                                selectedLesson === lIdx
                                  ? 'bg-blue-50 text-blue-700 font-bold'
                                  : 'hover:bg-slate-50'
                              }`}
                            >
                              {lesson.title}
```

```
                          </button>
                        ))}
                        <button
                          onClick={() => {
                            const newModules = [...webbook.modules];
                            newModules[mIdx].lessons.push({
                              id: Date.now().toString(),
                              title: `Lekcja ${module.lessons.length +
1}`,
                              subtitle: '',
                              content: '',
                              multimedia: {}
                            });
                            setWebbook({ ...webbook, modules:
newModules });
                          }}
                          className="w-full px-3 py-2 border-2 border-
dashed border-slate-300 rounded-lg text-sm text-slate-500 hover:border-
blue-500 hover:text-blue-500 transition-all"
                        >
                          + Dodaj Lekcję
                        </button>
                      </div>
                    )}
                  </div>
                ))}
              </div>
            </div>
          </div>
        </aside>

        <main className="flex-1 p-8 overflow-y-auto h-[calc(100vh-
73px)]">
          {selectedModule !== null && selectedLesson !== null ? (
            <LessonEditor

lesson={webbook.modules[selectedModule].lessons[selectedLesson]}
              onChange={(updated) => {
                const newModules = [...webbook.modules];
                newModules[selectedModule].lessons[selectedLesson] =
updated;
                setWebbook({ ...webbook, modules: newModules });
              }}
            />
          ) : (
            <div className="h-full flex items-center justify-center">
              <div className="text-center">
                <Book size={80} className="mx-auto mb-6 text-slate-300"
/>
                <p className="text-xl text-slate-500 mb-2 font-
bold">Wybierz lekcję z menu</p>
                <p className="text-slate-400">lub dodaj nowy
moduł/lekcję</p>
              </div>
```

```
              </div>
            )}
          </main>
        </div>
      </div>
    );
  }

  //
==========================================================================
===
  // PREVIEW
  //
==========================================================================
===

  if (currentView === 'preview') {
    return (
      <div className="min-h-screen bg-white">
        <header className="border-b border-slate-200 bg-white sticky top-
0 z-50">
          <div className="px-8 py-4 flex justify-between items-center">
            <button
              onClick={() => setCurrentView('builder')}
              className="px-4 py-2 bg-slate-100 hover:bg-slate-200
rounded-lg font-bold flex items-center gap-2"
            >
              <ChevronRight size={20} className="rotate-180" />
              Powrót
            </button>
            <h2 className="text-lg font-bold text-slate-600">Podgląd
Webbooka</h2>
          </div>
        </header>

        <div className="max-w-4xl mx-auto px-8 py-16">
          <div className="mb-16 text-center border-b-4 border-slate-900
pb-12">
            <h1 className="text-6xl font-black mb-
6">{webbook.metadata.title}</h1>
            <p className
 <p className="text-2xl text-slate-600">{webbook.metadata.subtitle}</p>
            {webbook.metadata.author && (
              <p className="text-xl text-slate-500 mt-4">Autor:
{webbook.metadata.author}</p>
            )}
          </div>

          {webbook.modules.map((module, mIdx) => (
            <div key={module.id} className="mb-16">
              <h2 className="text-4xl font-black mb-8 border-l-8 border-
blue-500 pl-6">
                {module.title}
              </h2>
```

```
              {module.lessons.map((lesson, lIdx) => (
                <div key={lesson.id} className="mb-12 bg-slate-50
rounded-2xl p-8">
                  <h3 className="text-2xl font-black mb-
4">{lesson.title}</h3>
                  {lesson.subtitle && (
                    <p className="text-lg text-slate-600 mb-
6">{lesson.subtitle}</p>
                  )}
                  <div className="prose max-w-none mb-6">
                    <p className="whitespace-pre-wrap leading-
relaxed">{lesson.content}</p>
                  </div>

                  {lesson.multimedia.images &&
lesson.multimedia.images.length > 0 && (
                    <div className="grid grid-cols-2 gap-4 mb-6">
                      {lesson.multimedia.images.map(img => (
                        <img key={img.id} src={img.url} alt={img.name}
className="w-full rounded-xl" />
                      ))}
                    </div>
                  )}

                  {lesson.multimedia.audio && (
                    <div className="bg-blue-50 p-6 rounded-xl mb-6">
                      <p className="font-bold mb-3">🎧
{lesson.multimedia.audio.name}</p>
                      <audio controls className="w-full"
src={lesson.multimedia.audio.url} />
                    </div>
                  )}

                  {lesson.multimedia.video && (
                    <div className="mb-6">
                      <iframe
                        width="100%"
                        height="400"

src={`https://www.youtube.com/embed/${lesson.multimedia.video}`}
                        className="rounded-xl"
                        allowFullScreen
                      />
                    </div>
                  )}

                  {lesson.worksheet && (
                    <div className="mt-8 bg-white rounded-xl p-6 border-2
border-slate-200">
                      <h4 className="font-bold text-lg mb-4">📋
{lesson.worksheet.title}</h4>
                      {lesson.worksheet.questions.map(q => (
                        <div key={q.id} className="mb-4">
```

```
                          <label className="block font-bold mb-
2">{q.question}</label>
                          {q.type === 'textarea' ? (
                            <textarea className="w-full p-3 border
rounded-lg" rows={4} />
                          ) : (
                            <input type="text" className="w-full p-3
border rounded-lg" />
                          )}
                        </div>
                      ))}
                    </div>
                  )}

                  {lesson.quiz && (
                    <div className="mt-8 bg-purple-50 rounded-xl p-6
border-2 border-purple-200">
                      <h4 className="font-bold text-lg mb-4">?
{lesson.quiz.title}</h4>
                      {lesson.quiz.questions.map((q, qIdx) => (
                        <div key={q.id} className="mb-6">
                          <p className="font-bold mb-3">{qIdx + 1}.
{q.question}</p>
                          <div className="space-y-2">
                            {q.options.map((opt, oIdx) => (
                              <label key={oIdx} className="flex items-
center gap-3 p-3 bg-white rounded-lg cursor-pointer hover:bg-purple-100
transition-all">
                                <input type="radio" name={`q-${qIdx}`} />
                                <span>{opt}</span>
                              </label>
                            ))}
                          </div>
                        </div>
                      ))}
                    </div>
                  )}
                </div>
              ))}
            </div>
          ))}
        </div>
      </div>
    );
  }

  return null;
};

export default App;
//
==========================================================================
===
// WEBBOOK GENERATOR
```

```
//
==========================================================================
===

function generateWebbook(webbook: Webbook): string {
  const modulesHTML = webbook.modules.map((module, mIdx) =>
    module.lessons.map((lesson, lIdx) => `
      <div class="page ${mIdx === 0 && lIdx === 0 ? 'active' : ''}"
id="page-${mIdx}-${lIdx}">
        <div class="page-header">
          <h1>${lesson.title}</h1>
          ${lesson.subtitle ? `<p
class="subtitle">${lesson.subtitle}</p>` : ''}
        </div>

        <div class="content">
          <p style="white-space: pre-wrap; line-height:
1.8;">${lesson.content}</p>
        </div>

        ${lesson.multimedia.images && lesson.multimedia.images.length > 0
? `
        <div class="media-gallery">
          ${lesson.multimedia.images.map(img => `
            <img src="${img.url}" alt="${img.name}" />
          `).join('')}
        </div>
        ` : ''}

        ${lesson.multimedia.audio ? `
        <div class="audio-player">
          <p class="audio-title">🎧 ${lesson.multimedia.audio.name}</p>
          <audio controls src="${lesson.multimedia.audio.url}"></audio>
        </div>
        ` : ''}

        ${lesson.multimedia.video ? `
        <div class="video-container">
          <iframe
src="https://www.youtube.com/embed/${lesson.multimedia.video}"
allowfullscreen></iframe>
        </div>
        ` : ''}

        ${lesson.worksheet ? `
        <div class="worksheet">
          <h3>${lesson.worksheet.title}</h3>
          ${lesson.worksheet.questions.map(q => `
            <div class="form-group">
              <label>${q.question}</label>
              ${q.type === 'textarea'
                ? '<textarea rows="4"></textarea>'
                : '<input type="text" />'
              }
```

```
            </div>
          `).join('')}
          <button class="btn btn-save" onclick="alert('Odpowiedzi
zapisane!')">Zapisz odpowiedzi</button>
        </div>
        ` : ''}

        ${lesson.quiz ? `
        <div class="quiz">
          <h3>${lesson.quiz.title}</h3>
          ${lesson.quiz.questions.map((q, qIdx) => `
            <div class="quiz-question">
              <p class="question-text">${qIdx + 1}. ${q.question}</p>
              <div class="quiz-options">
                ${q.options.map((opt, oIdx) => `
                  <label class="quiz-option">
                    <input type="radio" name="q${mIdx}-${lIdx}-${qIdx}"
value="${oIdx}" />
                    <span>${opt}</span>
                  </label>
                `).join('')}
              </div>
            </div>
          `).join('')}
          <button class="btn btn-primary" onclick="alert('Quiz
ukończony!')">Sprawdź odpowiedzi</button>
        </div>
        ` : ''}

        <div class="navigation-buttons">
          ${lIdx > 0 || mIdx > 0 ? `<button class="btn btn-secondary"
onclick="showPage('page-${lIdx > 0 ? mIdx : mIdx - 1}-${lIdx > 0 ? lIdx -
1 : webbook.modules[mIdx - 1]?.lessons.length - 1 || 0}')">←
Poprzednia</button>` : ''}
          ${lIdx < module.lessons.length - 1 || mIdx <
webbook.modules.length - 1 ? `<button class="btn btn-primary"
onclick="showPage('page-${lIdx < module.lessons.length - 1 ? mIdx : mIdx
+ 1}-${lIdx < module.lessons.length - 1 ? lIdx + 1 : 0}')">Następna
→</button>` : ''}
        </div>
      </div>
    `).join('')
  ).join('');

  const sidebarHTML = webbook.modules.map((module, mIdx) => `
    <div class="sidebar-section">
      <div class="sidebar-title">${module.title}</div>
      ${module.lessons.map((lesson, lIdx) => `
        <a href="#" class="sidebar-link ${mIdx === 0 && lIdx === 0 ?
'active' : ''}"
           onclick="showPage('page-${mIdx}-${lIdx}'); return false;">
          ${lesson.title}
        </a>
      `).join('')}
```

```
    </div>
  `).join('');

  return `<!DOCTYPE html>
<html lang="${webbook.metadata.language}">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>${webbook.metadata.title}</title>
  <meta name="description" content="${webbook.metadata.description ||
webbook.metadata.subtitle}">
  <meta name="author" content="${webbook.metadata.author}">
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body { font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI',
Roboto, sans-serif; background: #f5f5f5; line-height: 1.6; }

    .header {
      background: linear-gradient(135deg,
${webbook.branding.primaryColor}, ${webbook.branding.secondaryColor});
      color: white;
      padding: 1.5rem 2rem;
      position: sticky;
      top: 0;
      z-index: 100;
      box-shadow: 0 2px 10px rgba(0,0,0,0.1);
    }
    .logo { font-size: 1.5rem; font-weight: bold; }

    .container { display: flex; min-height: calc(100vh - 70px); }
    .sidebar {
      width: 280px;
      background: white;
      border-right: 1px solid #ddd;
      overflow-y: auto;
      position: sticky;
      top: 70px;
      height: calc(100vh - 70px);
    }
    .sidebar-section { padding: 1.5rem 0; border-bottom: 1px solid #eee;
}
    .sidebar-title {
      padding: 0 1.5rem;
      font-weight: bold;
      color: ${webbook.branding.primaryColor};
      font-size: 0.85rem;
      text-transform: uppercase;
      margin-bottom: 0.75rem;
    }
    .sidebar-link {
      display: block;
      padding: 0.75rem 1.5rem;
      color: #666;
      text-decoration: none;
```

```
      border-left: 3px solid transparent;
      transition: all 0.3s;
    }
    .sidebar-link:hover, .sidebar-link.active {
      background: #f0f7ff;
      border-left-color: ${webbook.branding.primaryColor};
      color: ${webbook.branding.primaryColor};
      font-weight: 500;
    }

    main { flex: 1; padding: 3rem 2rem; }
    .page { display: none; max-width: 900px; margin: 0 auto; animation:
fadeIn 0.3s; }
    .page.active { display: block; }
    @keyframes fadeIn { from { opacity: 0; transform: translateY(10px); }
to { opacity: 1; transform: translateY(0); } }

    .page-header { margin-bottom: 2rem; border-bottom: 2px solid #f0f0f0;
padding-bottom: 1.5rem; }
    .page-header h1 { font-size: 2.5rem; color: #1f2937; margin-bottom:
0.5rem; }
    .subtitle { color: #666; font-size: 1.1rem; }

    .content { line-height: 1.8; color: #333; margin-bottom: 2rem; }

    .media-gallery {
      display: grid;
      grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
      gap: 1rem;
      margin: 2rem 0;
    }
    .media-gallery img {
      width: 100%;
      height: 200px;
      object-fit: cover;
      border-radius: 12px;
      box-shadow: 0 4px 12px rgba(0,0,0,0.1);
    }

    .audio-player {
      background: #f0f7ff;
      padding: 1.5rem;
      border-radius: 12px;
      margin: 2rem 0;
      border: 1px solid #bfdbfe;
    }
    .audio-title { font-weight: 600; color:
${webbook.branding.primaryColor}; margin-bottom: 1rem; }
    .audio-player audio { width: 100%; margin-top: 0.5rem; }

    .video-container {
      position: relative;
      width: 100%;
      padding-bottom: 56.25%;
```

```
  height: 0;
  overflow: hidden;
  border-radius: 12px;
  margin: 2rem 0;
}
.video-container iframe {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  border: none;
}

.worksheet {
  background: white;
  padding: 2rem;
  border-radius: 12px;
  margin: 2rem 0;
  border: 2px solid ${webbook.branding.accentColor};
}
.worksheet h3 {
  color: ${webbook.branding.primaryColor};
  margin-bottom: 1.5rem;
  font-size: 1.5rem;
}
.form-group { margin-bottom: 1.5rem; }
.form-group label {
  display: block;
  font-weight: 600;
  margin-bottom: 0.5rem;
  color: #374151;
}
.form-group input, .form-group textarea {
  width: 100%;
  padding: 0.75rem;
  border: 1px solid #d1d5db;
  border-radius: 8px;
  font-family: inherit;
  font-size: 1rem;
}
.form-group textarea { resize: vertical; }

.quiz {
  background: #faf5ff;
  padding: 2rem;
  border-radius: 12px;
  margin: 2rem 0;
  border: 2px solid #c084fc;
}
.quiz h3 {
  color: #7c3aed;
  margin-bottom: 1.5rem;
  font-size: 1.5rem;
```

```
    }
    .quiz-question { margin-bottom: 2rem; }
    .question-text {
      font-weight: 600;
      margin-bottom: 1rem;
      color: #374151;
    }
    .quiz-options { display: flex; flex-direction: column; gap: 0.75rem;
}
    .quiz-option {
      display: flex;
      align-items: center;
      gap: 0.75rem;
      padding: 1rem;
      background: white;
      border: 2px solid #e9d5ff;
      border-radius: 8px;
      cursor: pointer;
      transition: all 0.3s;
    }
    .quiz-option:hover {
      background: #f3e8ff;
      border-color: #c084fc;
    }
    .quiz-option input[type="radio"] {
      width: 18px;
      height: 18px;
      cursor: pointer;
    }

    .btn {
      padding: 0.75rem 1.5rem;
      border: none;
      border-radius: 8px;
      font-weight: 600;
      cursor: pointer;
      transition: all 0.3s;
      font-size: 1rem;
    }
    .btn-primary {
      background: ${webbook.branding.primaryColor};
      color: white;
    }
    .btn-primary:hover {
      opacity: 0.9;
      transform: translateY(-2px);
    }
    .btn-secondary {
      background: #6b7280;
      color: white;
    }
    .btn-secondary:hover {
      background: #4b5563;
    }
```

```
    .btn-save {
      background: ${webbook.branding.accentColor};
      color: white;
      margin-top: 1rem;
    }

    .navigation-buttons {
      display: flex;
      gap: 1rem;
      justify-content: space-between;
      margin-top: 3rem;
      padding-top: 2rem;
      border-top: 2px solid #f0f0f0;
    }

    @media (max-width: 768px) {
      .container { flex-direction: column; }
      .sidebar {
        width: 100%;
        position: static;
        height: auto;
        border-right: none;
        border-bottom: 1px solid #ddd;
      }
      main { padding: 2rem 1rem; }
      .page-header h1 { font-size: 2rem; }
    }
  </style>
</head>
<body>
  <header class="header">
    <div class="logo">${webbook.metadata.title}</div>
  </header>

  <div class="container">
    <aside class="sidebar">
      ${sidebarHTML}
    </aside>

    <main>
      ${modulesHTML}
    </main>
  </div>

  <script>
    function showPage(pageId) {
      // Hide all pages
      document.querySelectorAll('.page').forEach(page => {
        page.classList.remove('active');
      });

      // Show selected page
      const page = document.getElementById(pageId);
      if (page) {
```

```
        page.classList.add('active');

        // Update sidebar active link
        document.querySelectorAll('.sidebar-link').forEach(link => {
          link.classList.remove('active');
        });

        const activeLink =
document.querySelector(\`[onclick="showPage('\${pageId}'); return
false;"]\`);
        if (activeLink) {
          activeLink.classList.add('active');
        }

        // Scroll to top
        window.scrollTo({ top: 0, behavior: 'smooth' });
      }
    }
  </script>
</body>
</html>`;
}
```

IMPLEMENTACJA KOMPLETNA!
Stworzyłem pełnowymiarowy DTR Studio - Professional Webbook Creator z następującymi funkcjami:

🎯 Główne Funkcje:

1. Ekran Startowy

✅ "Nowy Projekt" - zacznij od zera
✅ "Szablon" - wybierz gotową strukturę (7-dniowy, 21-dniowy, kurs modułowy)
✅ "Wczytaj" - otwórz zapisany projekt JSON
✅ Prezentacja możliwości (moduły, audio/video, karty, quizy)

2. Builder (Kreator)

✅ Sidebar - zarządzanie modułami i lekcjami
✅ Edytor lekcji z pełnym wsparciem dla:

Tekst (tytuł, podtytuł, treść)
Obrazy (upload, galeria, drag & drop)
Audio (upload plików audio)
Video (YouTube ID)
Karty pracy (pytania, krótkie/długie odpowiedzi)
Quizy (pytania wielokrotnego wyboru, poprawne odpowiedzi)

3. Ustawienia

✅ Branding - edycja kolorów (primary, secondary, accent)

✅ Color picker + hex input
✅ Podgląd kolorów na żywo
✅ Zapisz projekt (.json)

4. Podgląd

✅ Profesjonalny podgląd jak wygląda webbook
✅ Wszystkie moduły i lekcje
✅ Multimedia (obrazy, audio, video)
✅ Karty pracy i quizy

5. Eksport HTML

✅ Jeden klik → gotowy plik .html
✅ Pełna struktura:

Responsywny sidebar z nawigacją
Strony z treścią
Galerie obrazów
Odtwarzacze audio
Osadzone wideo YouTube
Karty pracy z formularzami
Quizy z radio buttons
Nawigacja (poprzednia/następna)

✅ Własne kolory z ustawień
✅ Responsywność (desktop + mobile)
✅ Smooth animations

📚 Gotowe Szablony:

7-Dniowy Kurs - tygodniowy program z codziennymi lekcjami
21-Dniowy Program - 3 tygodnie podzielone na części (fundamenty, praktyka, mistrzostwo)
Kurs Modułowy - 5 modułów po 3 lekcje każdy

🎨 Design:

Ciemny interfejs startowy (mniej męczący)
Jasny kreator (produktywność)
Profesjonalne kolory i animacje
Duże, czytelne przyciski
Intuicyjna nawigacja

📖 Jak Używać:

Wybierz "Nowy Projekt" lub "Szablon"
Wpisz dane webbooka (tytuł, autor)
Dodaj moduły i lekcje
Wypełnij treść każdej lekcji

Dodaj multimedia (obrazy, audio, video)
Stwórz karty i quizy (opcjonalnie)
Ustaw kolory w Ustawieniach
Kliknij "Podgląd" aby zobaczyć efekt
Kliknij "Eksport HTML" aby pobrać gotowy plik
🚀 Export:
Wygenerowany plik HTML jest gotowy do użycia:

Otwórz w przeglądarce
Hostuj na serwerze
Konwertuj na PDF/EPUB
Wszystko w jednym pliku!